

FORMATION EMBARQUÉE

Module 00 — Fondamentaux

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 00 — Fondamentaux : ce que tout programmeur embarqué doit savoir

Objectif : comprendre ce qui se passe *physiquement* quand un programme s'exécute. Sans ça, les pointeurs, les registres et le VHDL resteront de la magie noire.

1. Le binaire et l'hexadécimal

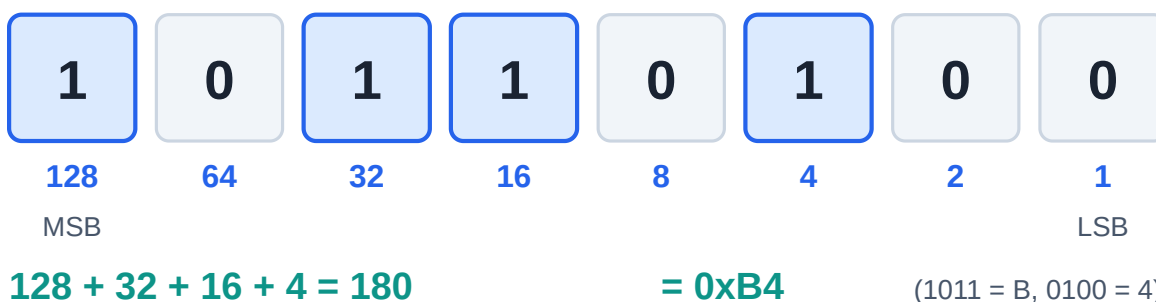
Un ordinateur ne connaît que deux états : tension présente (1) ou absente (0). Tout — nombres, textes, images, instructions — est codé en binaire.

1.1 Compter en binaire

Chaque position vaut une puissance de 2 (comme chaque position vaut une puissance de 10 en décimal) :

```
binaire : 1 0 1 1
poids   : 8 4 2 1      → 8 + 0 + 2 + 1 = 11 en décimal
```

Nombre binaire 1011 0100 sur 8 bits



Poids des bits sur un octet et conversion en hexadécimal

Décimal	Binaire (4 bits)	Hexadécimal
0	0000	0x0
5	0101	0x5
10	1010	0xA
15	1111	0xF
255	1111 1111	0xFF

1.2 L'hexadécimal : le binaire pour humains

4 bits = 1 chiffre hexa (0–F). On écrit `0xFF` plutôt que `11111111`. En embarqué, **tout** se lit en hexa : adresses mémoire (`0x40021000`), valeurs de registres (`0x0000000F`), trames de communication.

1.3 Vocabulaire

- **bit** : un 0 ou un 1.
- **octet (byte)** : 8 bits. Valeurs de 0 à 255 (`0x00` à `0xFF`).
- **mot (word)** : dépend du processeur — 16, 32 ou 64 bits.
- **LSB / MSB** : bit de poids faible / fort.
- **endianness** : ordre des octets en mémoire. *Little-endian* (x86, la plupart des ARM) = octet de poids faible en premier.

1.4 Nombres signés : le complément à deux

Pour représenter les négatifs, on utilise le complément à deux : on inverse tous les bits puis on ajoute 1.

```
+5 sur 8 bits : 0000 0101
-5 sur 8 bits : 1111 1011    (inversion → 1111 1010, +1 → 1111 1011)
```

Sur 8 bits signés : plage de -128 à +127. Sur 8 bits non signés : 0 à 255. **Piège classique** : `uint8_t x = 0; x--;` donne 255, pas -1 (débordement).

Exercices

1. Convertis 42, 100 et 200 en binaire puis en hexa.
2. Que vaut `0xB7` en décimal ? En binaire ?
3. Écris -10 en complément à deux sur 8 bits.

2. Électronique numérique de base

2.1 Les portes logiques

Toute la logique d'un processeur est faite de portes :

Porte	Symbole C	Sortie = 1 si...
NOT (NON)	<code>!</code> , <code>~</code>	l'entrée est 0
AND (ET)	<code>&&</code> , <code>&</code>	toutes les entrées sont 1
OR (OU)	<code>& &</code> , <code> </code>	au moins une entrée est 1
XOR (OU exclusif)	<code>^</code>	les entrées sont différentes
NAND / NOR	<code>—</code>	l'inverse de AND / OR

La NAND est « universelle » : on peut construire n'importe quel circuit avec uniquement des NAND.

2.2 Logique combinatoire vs séquentielle

- **Combinatoire** : la sortie ne dépend *que* des entrées actuelles (additionneur, multiplexeur, décodeur). Pas de mémoire.
- **Séquentielle** : la sortie dépend aussi de l'état *passé* → il y a de la **mémoire**. La brique de base est la **bascule D (D flip-flop)** : elle recopie son entrée D sur sa sortie Q *au front montant de l'horloge*.

Cette distinction est la clé du VHDL (module 04) : on y décrit soit des blocs combinatoires, soit des registres cadencés par une horloge.

2.3 L'horloge (clock)

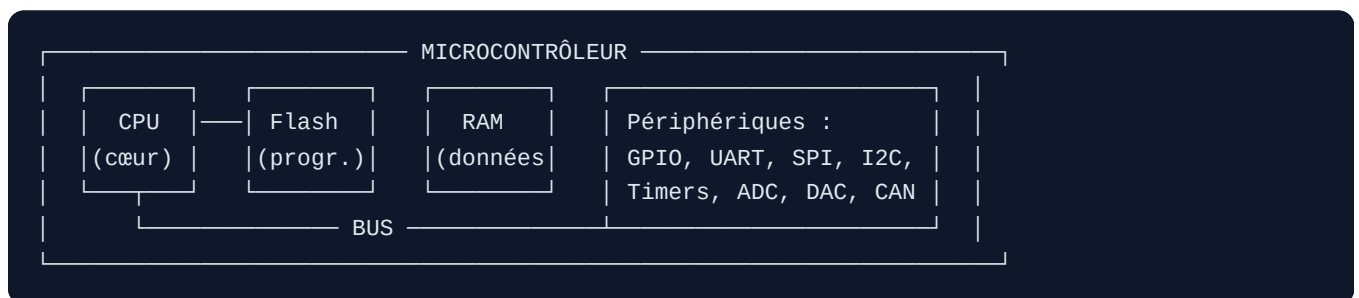
Un signal carré qui rythme tout le circuit. Un microcontrôleur à 16 MHz exécute son cycle de base 16 millions de fois par seconde. Sur FPGA, on raisonne en « que se passe-t-il à chaque front montant ? ».

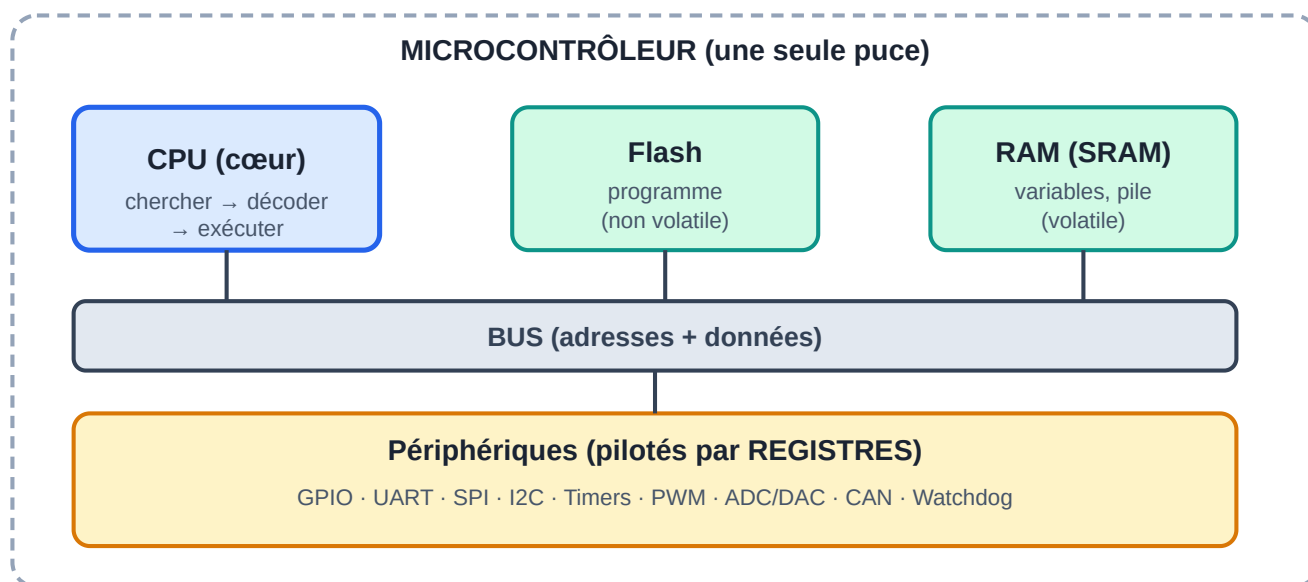
2.4 Niveaux logiques et le monde réel

- TTL 5 V (Arduino Uno) : 0 logique < 0,8 V ; 1 logique > 2 V.
- 3,3 V (STM32, ESP32, Raspberry Pi) : **ne jamais** injecter du 5 V sur une broche 3,3 V.
- **Résistance de pull-up / pull-down** : force un niveau connu quand rien ne pilote la ligne (indispensable pour les boutons et le bus I2C).
- **Rebond (bounce)** : un bouton mécanique génère des dizaines de transitions parasites pendant quelques ms. À filtrer en logiciel (attente ~20 ms) ou en matériel (condensateur).

3. Architecture d'un ordinateur / microcontrôleur

3.1 Les blocs





Blocs d'un microcontrôleur reliés par le bus

- **CPU** : exécute les instructions (chercher → décoder → exécuter).
- **Flash** : mémoire non volatile où vit le programme (survit à la coupure).
- **RAM (SRAM)** : mémoire volatile pour les variables, la pile, le tas.
- **Périphériques** : blocs matériels pilotés par des **registres**.

3.2 Microprocesseur vs microcontrôleur vs FPGA

	Microprocesseur (PC, Raspberry Pi)	Microcontrôleur (Arduino, STM32)	FPGA
Contient	CPU seul (RAM/flash externes)	CPU + RAM + flash + périphériques sur une puce	Matrice de logique reconfigurable
OS	Linux/Windows	Souvent aucun (bare-metal) ou RTOS	Aucun — c'est du matériel
Force	Puissance	Coût, temps réel, consommation	Parallélisme massif, latence nulle
Programmé en	C/C++/Python/Java	C/C++	VHDL/Verilog

3.3 Les registres : le cœur du bas niveau

Un **registre de périphérique** est une case mémoire spéciale : écrire dedans agit *physiquement* sur le matériel. Exemple (AVR/Arduino Uno) :

```

DDRB |= (1 << 5); // bit 5 de DDRB à 1 → broche PB5 en sortie
PORTB |= (1 << 5); // bit 5 de PORTB à 1 → PB5 à l'état haut → LED allumée
PORTB &= ~(1 << 5); // bit 5 à 0 → LED éteinte

```

C'est exactement ce que fait `digitalWrite()` d'Arduino sous le capot — en ~50 fois plus lent, à cause des vérifications qu'il ajoute.

3.4 Carte mémoire (memory map)

Toutes les adresses vivent dans un seul espace. Exemple typique ARM Cortex-M :

```
0x0000 0000 – Flash (code)
0x2000 0000 – SRAM (variables, pile)
0x4000 0000 – Périphériques (GPIO, UART, timers...)
0xE000 0000 – Registres système (NVIC, SysTick)
```

« Écrire à l'adresse `0x40021018` » peut vouloir dire « activer l'horloge du port GPIO C ». D'où l'importance des pointeurs en C.

3.5 Pile (stack) et tas (heap)

- **Pile** : variables locales, adresses de retour. Gérée automatiquement, rapide, mais petite (souvent quelques Ko). Le **débordement de pile** est le bug embarqué classique.
- **Tas** : allocation dynamique (`malloc` / `new`). En embarqué critique on l'évite (fragmentation, imprévisibilité) : tout est alloué statiquement.

4. Les périphériques essentiels

4.1 GPIO (General Purpose Input/Output)

Broche configurable en entrée ou sortie numérique. Modes usuels : sortie push-pull, sortie open-drain, entrée flottante, entrée avec pull-up.

4.2 Timers / compteurs

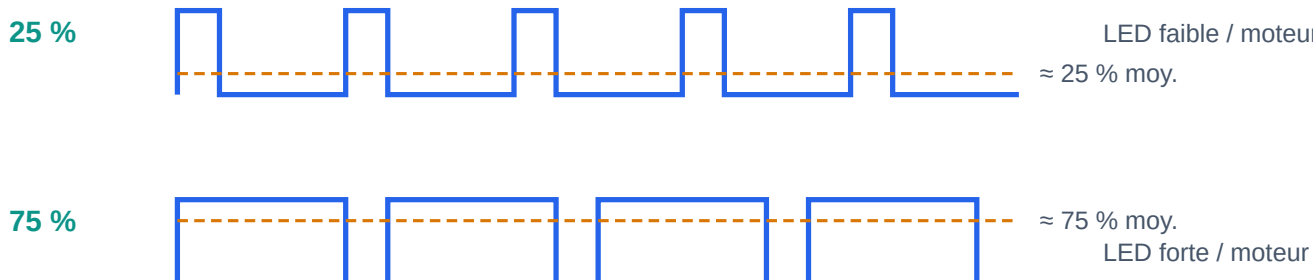
Compteurs matériels qui s'incrémentent à chaque tick d'horloge. Servent à : - mesurer le temps précisément, - générer des interruptions périodiques, - produire du **PWM** (voir ci-dessous), - compter des impulsions externes (codeur incrémental).

4.3 PWM (Pulse Width Modulation)

On fait varier la *puissance moyenne* en découpant un signal carré :

```
Rapport cyclique 25 % :  → LED faible, moteur lent
Rapport cyclique 75 % :  → LED forte, moteur rapide
```

PWM — la puissance moyenne varie avec le rapport cyclique



Même fréquence, seule la largeur d'impulsion change. L'œil et le moteur « intègrent » → luminosité / vitesse prop

PWM : le rapport cyclique fixe la puissance moyenne

Utilisé pour : luminosité de LED, vitesse de moteur, servo-moteurs (impulsion de 1 à 2 ms toutes les 20 ms), génération audio simple.

4.4 ADC / DAC

- **ADC** (Analog → Digital) : convertit une tension en nombre. Arduino Uno : 10 bits → 0–1023 pour 0–5 V. Résolution = $5\text{ V} / 1024 \approx 4,9\text{ mV}$.
- **DAC** (Digital → Analog) : l'inverse (rare sur les petits micros ; on émule avec du PWM + filtre RC).

4.5 Watchdog

Compteur qui redémarre le micro si le programme ne le « caresse » pas régulièrement. Filet de sécurité contre les plantages — obligatoire dans les produits industriels.

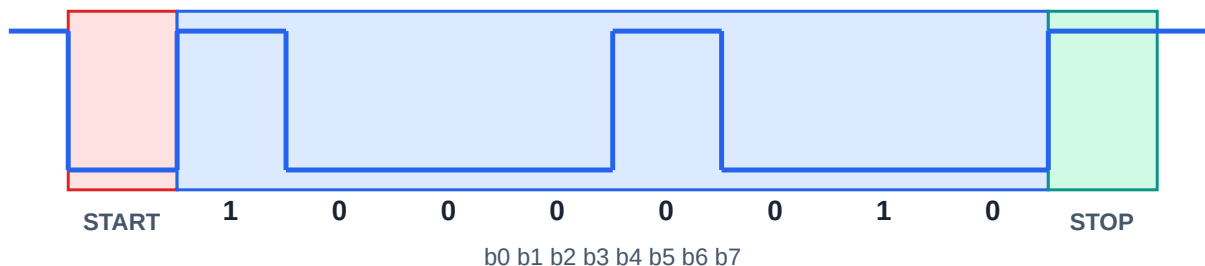
5. Les protocoles de communication

C'est LE sujet qui revient dans tous les entretiens embarqués.

5.1 UART (liaison série asynchrone)

- 2 fils : TX (émission) et RX (réception), croisés entre les deux équipements.
- Pas d'horloge partagée → les deux côtés doivent convenir d'un **baudrate** (9600, 115200 bauds...).
- Trame : bit de start, 8 bits de données, (parité), bit de stop.
- C'est ce qu'utilise `Serial.begin(9600)` sur Arduino.
- Variantes industrielles : **RS-232** ($\pm 12\text{ V}$, point à point), **RS-485** (différentiel, multipoint, base du Modbus RTU — voir module 08).

Trame UART 8N1 — caractère 'A' = 0x41 = 0100 0001 (LSB d'abord)



t (bas) · 8 bits de données · 1 bit de stop (haut)

μs, trame ≈ 1,04 ms → 960 octets/s max

Trame UART 8N1 pour le caractère 'A'

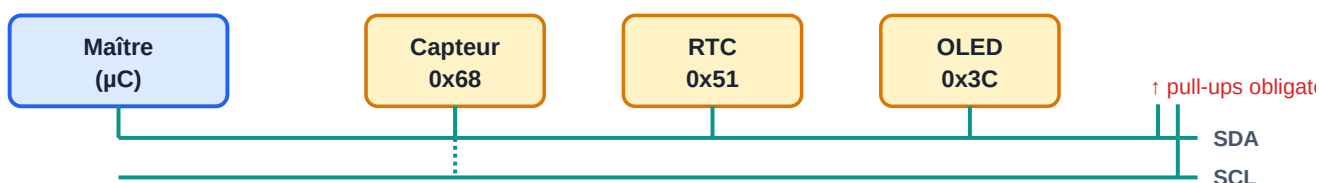
5.2 I2C (Inter-Integrated Circuit)

- 2 fils partagés par tous : **SDA** (données) + **SCL** (horloge), avec pull-ups.
- Un maître, plusieurs esclaves, chacun a une **adresse 7 bits** (ex. 0x68 pour le MPU-6050).
- Vitesse : 100 kHz / 400 kHz. Idéal pour capteurs, écrans OLED, RTC.
- Séquence : START → adresse + bit R/W → ACK → données → STOP.

5.3 SPI (Serial Peripheral Interface)

- 4 fils : **MOSI**, **MISO**, **SCK** (horloge), **CS** (sélection, un par esclave).
- Full duplex, très rapide (dizaines de MHz). Idéal pour cartes SD, écrans TFT, mémoires flash.
- Paramètres : polarité et phase d'horloge (modes 0 à 3).

I2C — 2 fils partagés (SDA + SCL), adressage 7 bits



SPI — full duplex, 1 fil CS par esclave (rapide : dizaines de MHz)



Topologies I2C (bus partagé) et SPI (un CS par esclave)

5.4 CAN (Controller Area Network)

- Bus différentiel 2 fils, multi-maître, très robuste. Standard de l'automobile et de beaucoup d'industriel.

- Trames courtes (8 octets classiques) avec identifiant qui sert de priorité (arbitrage sans collision).

5.5 Réseaux industriels (aperçu, détaillé aux modules 07-08)

- **Modbus RTU/TCP** : simple, vénérable, partout (Schneider en est l'origine).
- **PROFINET / PROFIBUS** : monde Siemens.
- **EtherCAT, CANopen, IO-Link** : motion et capteurs intelligents.

Comparatif rapide

	UART	I2C	SPI	CAN
Fils	2	2	4+	2
Vitesse	~1 Mbps	0,4 Mbps	50+ Mbps	1 Mbps
Multi-équipements	non	oui (adresses)	oui (1 CS chacun)	oui
Usage type	debug, GPS, modem	capteurs	écrans, SD	auto, industrie

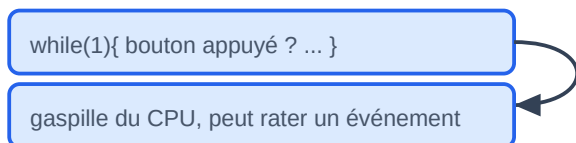
6. Interruptions et temps réel

6.1 Polling vs interruption

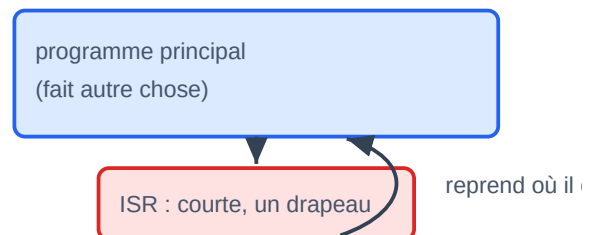
- **Polling** : la boucle principale vérifie sans cesse « le bouton est-il appuyé ? ». Simple, mais gaspille du CPU et peut rater des événements.
- **Interruption (IRQ)** : le matériel *suspend* le programme, exécute une fonction spéciale (**ISR**, Interrupt Service Routine), puis reprend où il en était.

Polling vs Interruption

Polling — la boucle DEMANDE sans cesse



Interruption — le matériel PRÉVIENT



Règle d'or : une ISR lève un drapeau (volatile !), stocke une valeur, et sort. Jamais de `delay()`, de `printf`, d'allocation.

Polling contre interruption

6.2 Règles d'or des ISR

1. **Courtes** : lever un drapeau, stocker une valeur, sortir. Jamais de `delay()`, de `printf`, d'allocation.
2. Les variables partagées entre ISR et boucle principale doivent être **volatile** (voir module 01) et, si > 1 octet, protégées (section critique : désactiver brièvement les interruptions).

3. Attention aux priorités : une ISR peut en interrompre une autre (nesting).

6.3 Temps réel

« Temps réel » ≠ « rapide ». Cela signifie **garantie de délai** : la réponse arrive *toujours* avant l'échéance. Un airbag qui répond en 1 s est rapide à l'échelle humaine mais inutile. D'où les **RTOS** (FreeRTOS, Zephyr — module 06) qui ordonnent des tâches par priorité de façon déterministe.

7. La chaîne de compilation (toolchain)

Que se passe-t-il entre ton `.c` et la puce qui clignote ?

```
main.c —[préprocesseur]—> main.i    (macros et #include expansés)
      —[compilateur]——> main.s    (assembleur)
      —[assembleur]——> main.o    (code machine, adresses non résolues)
      —[éditeur de liens]—> firmware.elf (tout assemblé, adresses fixées
                                         par le script de linkage .ld)
      —[objcopy]——> firmware.bin/.hex (image brute à flasher)
      —[programmeur]——> la puce (via USB, ST-Link, JTAG/SWD...)
```

Chaîne de compilation : du code source à la puce



Cross-compilation : on compile sur PC (x86) POUR une autre cible (ARM, AVR).
`arm-none-eabi-gcc` · `avr-gcc` · le programmeur écrit l'image dans la Flash.

La chaîne de compilation, du source à la puce

- **Cross-compilation** : on compile sur PC (x86) pour une autre cible (ARM) : `arm-none-eabi-gcc` , `avr-gcc` .
- **Débogueur matériel** (JTAG/SWD + GDB/OpenOCD) : points d'arrêt et inspection des variables sur la puce elle-même. Bien plus puissant que le `printf` de debug — mais apprend les deux.

8. Ce qu'il faut retenir

- Tout est binaire ; l'hexa est ta langue de lecture.
- Un microcontrôleur = CPU + mémoires + périphériques pilotés par registres.
- Écrire un bit dans un registre = agir sur le monde physique.
- UART / I2C / SPI / CAN : connais leurs fils, leurs forces, leurs usages.

- Interruptions courtes, variables partagées `volatile` .
- La toolchain transforme ton C en bits dans la flash.

➡ Passe au **Module 01 — Langage C** : c'est là que tout devient concret.

FORMATION EMBARQUÉE

Module 01 — Langage C

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 01 — Le langage C : la langue maternelle de l'embarqué

Le C est le langage des firmwares, des drivers, des OS et des RTOS. Il est petit (une trentaine de mots-clés), proche du matériel, et n'a presque rien caché sous le capot. Le maîtriser, c'est comprendre *tout* le reste.

Pour pratiquer sans matériel : installe GCC (`sudo apt install build-essential` sous Linux, MinGW/WSL sous Windows) ou utilise <https://www.onlinegdb.com>.

```
gcc main.c -o main -Wall -Wextra  # compile avec tous les avertissements
./main                             # exécute
```

1. Premier programme et structure

```
#include <stdio.h>           // directive préprocesseur : inclut les déclarations d'E/S

int main(void) {             // point d'entrée : le programme commence ici
    printf("Bonjour\n");     // affiche du texte
    return 0;                // 0 = succès
}
```

En embarqué bare-metal, il n'y a pas de `printf` par défaut ni de retour de `main` : le programme est une boucle infinie.

```
int main(void) {
    init_hardware();
    while (1) {               // boucle principale – ne se termine jamais
        // lire capteurs, décider, agir
    }
}
```

2. Types et variables

2.1 Types de base — et pourquoi on ne les utilise pas en embarqué

`int`, `short`, `long` ont une taille **qui dépend de la machine** (un `int` fait 16 bits sur AVR, 32 sur ARM). En embarqué on veut du précis, donc on utilise `<stdint.h>` :

```
#include <stdint.h>

uint8_t  compteur = 0;          // entier non signé 8 bits  (0 à 255)
int16_t  temperature = -40;    // entier signé 16 bits   (-32768 à 32767)
uint32_t adresse = 0x40021000; // 32 bits non signé
float    tension = 3.3f;       // flottant 32 bits (coûteux sans FPU !)
```

Règles embarqué : - Toujours la taille explicite (`uint8_t` , pas `unsigned char`). - Éviter `float` / `double` sur les petits micros sans unité flottante : préférer l'arithmétique en **point fixe** (ex. stocker des millivolts en `int32_t` plutôt que des volts en `float`).

2.2 Constantes

```
#define LED_PIN 5                // macro préprocesseur (texte remplacé)
const uint32_t BAUDRATE = 115200; // constante typée (préférable)
enum { MAX_CAPTEURS = 8 };       // constante entière de compilation
```

2.3 Portée et durée de vie

```
uint8_t globale;                // globale : existe tout le programme, visible partout
static uint8_t privee;          // static globale : visible dans CE fichier seulement

void f(void) {
    uint8_t locale = 0;         // sur la pile, détruite à la sortie de f
    static uint8_t persistante = 0; // conserve sa valeur entre les appels
    persistante++;
}
```

`static` a deux sens (piège d'entretien) : limiter la visibilité d'un symbole global au fichier, ou rendre persistante une variable locale.

3. Opérateurs — dont les indispensables bit à bit

3.1 Classiques

`+` `-` `*` `/` `%` , comparaisons `==` `!=` `<` `>` , logiques `&&` `||` `!` , affectations composées `+=` `-=` `<<=` .

Piège n°1 du C : `=` (affectation) vs `==` (comparaison). `if (x = 5)` compile et est toujours vrai. Active `-Wall` !

3.2 Bit à bit : le pain quotidien de l'embarqué

```
uint8_t r = 0;

r |= (1 << 3);      // METTRE le bit 3 à 1      → 0000 1000
r &= ~(1 << 3);     // METTRE le bit 3 à 0      → 0000 0000
r ^= (1 << 3);      // INVERSER le bit 3 (toggle)
if (r & (1 << 3)) {} // TESTER le bit 3

uint8_t bas = octet & 0x0F;      // garder les 4 bits de poids faible (masque)
uint8_t haut = (octet >> 4) & 0x0F; // extraire les 4 bits de poids fort

uint16_t mot = ((uint16_t)hi << 8) | lo; // assembler 2 octets en 16 bits
```

Les 4 opérations de bits fondamentales (bit 3 ciblé)

SET <code>r = (1<<3)</code>	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	1	0	0	0	bit 3 → 1
0	0	0	0	1	0	0	0			
CLEAR <code>r &= ~(1<<3)</code>	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	1	1	1	bit 3 → 0
1	1	1	1	0	1	1	1			
TOGGLE <code>r ^= (1<<3)</code>	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>↑</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	↑	0	0	0	bit 3 inversé
0	0	0	0	↑	0	0	0			
TEST <code>r & (1<<3)</code>	<table><tr><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td><td>?</td></tr></table>	?	?	?	?	?	?	?	?	lit le bit 3 (≠0 si à 1)
?	?	?	?	?	?	?	?			

Les quatre opérations de bits : set, clear, toggle, test

Apprends ces cinq motifs par cœur (set, clear, toggle, test, masque). Ils servent dans chaque driver, chaque registre, chaque trame.

3.3 Décalages

`x << n` multiplie par 2^n , `x >> n` divise par 2^n . Décaler un signé négatif à droite est défini par l'implémentation → faire les manipulations de bits sur des **non signés**.

4. Contrôle de flux

```
if (temp > 30) { ventilo_on(); }
else if (temp > 25) { ventilo_mi_vitesse(); }
else { ventilo_off(); }

switch (etat) {                                // idéal pour les machines d'états
case ETAT_REPOS:
    /* ... */
    break;                                    // sans break, on TOMBE dans le cas suivant !
case ETAT_MESURE:
    /* ... */
    break;
default:
    erreur();
}

for (uint8_t i = 0; i < 8; i++) { /* 8 tours */ }
while (!(UCSR0A & (1 << UDRE0))) { } // attendre qu'un drapeau matériel se lève
do { c = lire(); } while (c != '\n'); // au moins un tour
```

5. Fonctions

```
// Prototype (déclaration) – dans le .h
int16_t moyenne(const int16_t *valeurs, uint8_t n);

// Définition – dans le .c
int16_t moyenne(const int16_t *valeurs, uint8_t n) {
    int32_t somme = 0;                        // 32 bits pour éviter le débordement
    for (uint8_t i = 0; i < n; i++) somme += valeurs[i];
    return (int16_t)(somme / n);
}
```

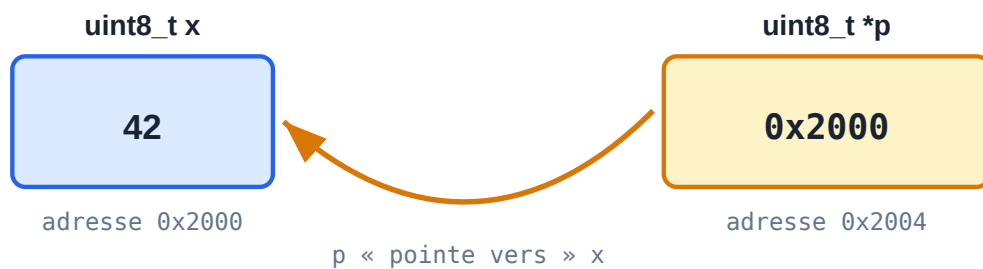
- Passage **par valeur** : la fonction reçoit une copie.
- Pour modifier une variable de l'appelant ou passer un tableau : **pointeur**.
- `const` sur un paramètre pointeur = « je promets de ne pas modifier ».

6. Les pointeurs — LE chapitre à maîtriser

Un pointeur est une variable qui contient une **adresse mémoire**.

```
uint8_t x = 42;
uint8_t *p = &x; // p contient l'ADRESSE de x (& = "adresse de")
uint8_t v = *p;  // v vaut 42 (* = "valeur pointée")
*p = 100;        // x vaut maintenant 100
```

Un pointeur contient l'ADRESSE d'une autre variable



```
uint8_t *p = &x; // p reçoit l'adresse de x
*p = 100; // écrit 100 DANS x (déréférencement)
```

Un pointeur contient l'adresse d'une variable

6.1 Pointeurs et tableaux

```
uint8_t tab[4] = {10, 20, 30, 40};
uint8_t *p = tab; // un tableau "se dégrade" en pointeur sur son 1er élément
p[2] == 30; // vrai
*(p+2) == 30; // strictement équivalent
```

L'arithmétique de pointeurs avance par *taille d'élément* : si `p` est un `uint32_t*`, `p+1` avance de 4 octets.

6.2 Pointeurs vers registres matériels

Voilà pourquoi l'embarqué adore les pointeurs — accéder à une adresse fixe :

```
#define GPIOC_ODR (*(volatile uint32_t *)0x4001100C)

GPIOC_ODR |= (1 << 13); // écrit dans le registre de sortie du port C
```

Décomposition : `0x4001100C` est un entier → casté en « pointeur vers `uint32_t` volatile » → déréférencé avec `*` → on obtient une lvalue qui EST le registre.

6.3 Pointeurs de fonction

```
void (*callback)(uint8_t) = NULL; // pointeur vers fonction(uint8_t)→void

void sur_reception(uint8_t octet) { /* ... */ }

callback = sur_reception;
if (callback) callback(0x55); // appel via le pointeur
```

Base des **callbacks**, des tables d'interruptions et des drivers génériques.

6.4 Les pièges mortels

```
uint8_t *p;           // NON INITIALISÉ → pointe n'importe où
*p = 5;               // ⚡ comportement indéfini

uint8_t *f(void) {
    uint8_t local = 3;
    return &local;    // ⚡ adresse d'une variable détruite à la sortie
}

char buf[8];
strcpy(buf, "trop long pour huit"); // ⚡ débordement de tampon
```

Réflexes : initialiser à `NULL` , tester avant usage, utiliser `snprintf` et les versions bornées, connaître la durée de vie de ce qu'on pointe.

7. Tableaux, chaînes, structures

7.1 Chaînes C

Une chaîne = tableau de `char` terminé par `'\0'` .

```
char nom[16] = "STM32";    // {'S','T','M','3','2','\0', ...}
strlen(nom);              // 5 (ne compte pas le \0)
strncpy(dst, src, sizeof dst - 1); // copie bornée – toujours borner !
```

7.2 Structures

```
typedef struct {
    uint8_t id;
    int16_t temperature_dixiemes; // 253 = 25,3 °C (point fixe !)
    uint32_t horodatage;
} Measure;

Measure m = { .id = 1, .temperature_dixiemes = 253, .horodatage = 0 };
m.id = 2;

Measure *pm = &m;
pm->id = 3; // flèche = déréférencement + accès champ
```

Alignement/padding : le compilateur insère des trous pour aligner les champs. `sizeof(Measure)` peut valoir 8 et non 7. Pour des trames réseau : `__attribute__((packed))` (GCC) — au prix d'accès plus lents.

7.3 Champs de bits et unions

```
typedef union {
    uint8_t octet;
    struct {
        uint8_t en_marche : 1;    // 1 bit
        uint8_t erreur     : 1;
        uint8_t mode       : 2;    // 2 bits
        uint8_t reserve    : 4;
    } bits;
} Statut;

Statut s = { .octet = 0 };
s.bits.mode = 2;
```

Pratique pour décoder des registres — mais l'ordre des bits dépend du compilateur : pour du code portable, préférer masques et décalages.

8. `volatile`, `const` et qualificateurs critiques

8.1 `volatile` — le mot-clé de l'embarqué

Dit au compilateur : « cette variable peut changer à tout moment en dehors du flot normal — ne mets pas sa valeur en cache, relis-la à chaque fois ».

Obligatoire pour : 1. les **registres matériels** (le matériel les modifie), 2. les variables partagées avec une **ISR**, 3. les variables partagées entre threads/tâches RTOS.

```
volatile uint8_t drapeau_rx = 0;    // écrit par l'ISR UART

// ISR
void USART_RX_vect(void) { drapeau_rx = 1; }

// boucle principale
while (!drapeau_rx) { }    // sans volatile, le compilateur optimiserait en boucle infinie
```

⚠ `volatile` ne rend PAS l'accès atomique. Un `uint32_t` partagé sur un CPU 8 bits se lit en 4 opérations : protéger par une section critique.

8.2 Lecture des déclarations complexes

```
const uint8_t *p;    // pointeur vers un octet constant (données protégées)
uint8_t *const p;    // pointeur constant vers un octet (adresse figée)
volatile uint32_t *reg; // pointeur vers registre volatile (le cas standard)
```

Astuce : lire de droite à gauche en partant du nom.

9. Mémoire dynamique — et pourquoi l'embarqué s'en méfie

```
uint8_t *buf = malloc(128);
if (buf == NULL) { /* gérer l'échec ! */ }
free(buf);
buf = NULL;           // évite le "dangling pointer"
```

Problèmes en embarqué : fragmentation du tas, échec imprévisible, fuites indétectables sur un système qui tourne des années. Règle courante (MISRA C, code aéronautique) : **aucune allocation dynamique après l'initialisation**. On dimensionne tout statiquement :

```
static uint8_t tampon_rx[256];           // alloué une fois pour toutes
```

10. Préprocesseur et organisation multi-fichiers

```
// capteur.h – l'interface publique
#ifndef CAPTEUR_H           // garde d'inclusion : évite la double inclusion
#define CAPTEUR_H
#include <stdint.h>

void capteur_init(void);
int16_t capteur_lire(void);

#endif
```

```
// capteur.c – l'implémentation
#include "capteur.h"

static uint8_t etat_interne; // static : invisible hors de ce fichier

void capteur_init(void) { /* ... */ }
int16_t capteur_lire(void) { /* ... */ return 0; }
```

Macros utiles et pièges :

```
#define MIN(a, b) ((a) < (b) ? (a) : (b)) // TOUJOURS parenthéser
#ifdef DEBUG
#define LOG(msg) uart_print(msg)
#else
#define LOG(msg)           // disparaît en production
#endif
```

Compilation de plusieurs fichiers : `gcc main.c capteur.c -o app`, ou mieux, un **Makefile** (module 06).

11. Exemple complet : blink bare-metal AVR (Arduino Uno sans Arduino)

```
#include <avr/io.h>           // définitions des registres de l'ATmega328P
#include <util/delay.h>

int main(void) {
    DDRB |= (1 << DDB5);      // PB5 (LED carte) en sortie

    while (1) {
        PORTB ^= (1 << PORTB5); // inverser la LED
        _delay_ms(500);
    }
}
```

Compilation et flash (avec `avr-gcc` + `avrdude`) :

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os blink.c -o blink.elf
avr-objcopy -O ihex blink.elf blink.hex
avrdude -c arduino -p m328p -P /dev/ttyACM0 -U flash:w:blink.hex
```

Tu viens de faire exactement ce que fait l'IDE Arduino — sans lui.

12. Machine d'états : le patron de conception n°1 de l'embarqué

Presque tout firmware est une machine d'états finis (FSM) — tu retrouveras ce même diagramme en C++, en VHDL et en GRAFCET (modules 02, 04, 07) :

Machine à états — feu tricolore (le motif récurrent du cours)

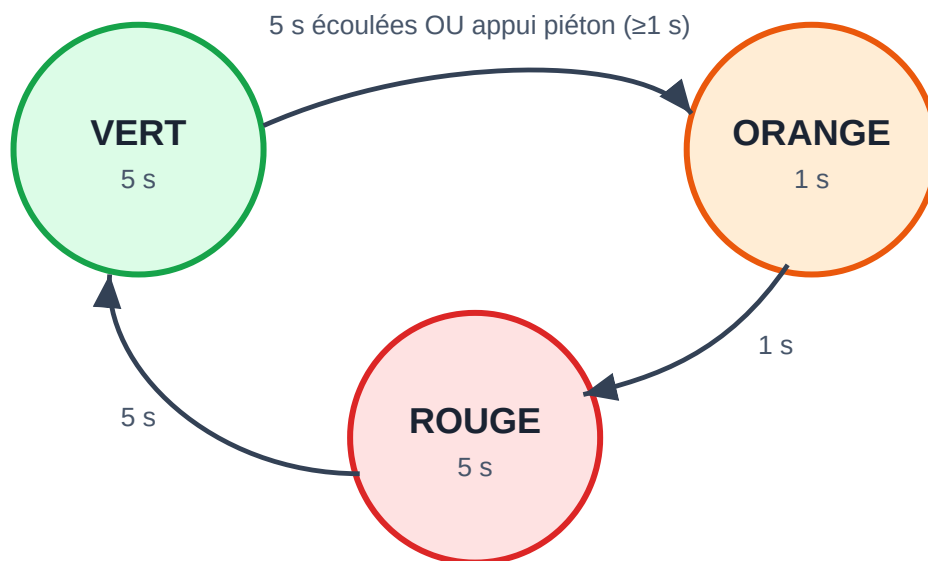


Diagramme d'états d'un feu tricolore

```

typedef enum { REPOS, CHAUFFE, MAINTIEN, ERREUR } Etat;

static Etat etat = REPOS;

void fsm_tick(void) {
    switch (etat) {
        case REPOS:
            if (bouton_marche()) { chauffage_on(); etat = CHAUFFE; }
            break;
        case CHAUFFE:
            if (temperature() >= CONSIGNE) etat = MAINTIEN;
            if (temperature() > MAXI) etat = ERREUR;
            break;
        case MAINTIEN:
            regulier();
            if (bouton_arret()) { chauffage_off(); etat = REPOS; }
            break;
        case ERREUR:
            chauffage_off();
            alarme();
            break;
    }
}

```

Tu retrouveras cette structure partout : en C, en C++, en VHDL (module 04) et dans le GRAFCET des automates (modules 07-08). C'est le même concept.

13. Bonnes pratiques embarqué (résumé MISRA-esque)

1. Compiler avec `-Wall -Wextra -Werror` et corriger *tous* les warnings.
2. Types explicites (`stdint.h`), pas de `int` nu.
3. Pas de `malloc` après l'init ; tailles bornées partout.
4. `volatile` sur tout ce qui est partagé avec le matériel ou une ISR.
5. ISR courtes ; drapeaux plutôt que traitement.
6. Une fonction = une responsabilité, < ~50 lignes.
7. Vérifier chaque valeur de retour (surtout les E/S).
8. Pas de récursion (pile bornée), pas de boucle sans limite de temps.
9. Tester les cas limites : 0, maximum, débordement, valeur négative.

Exercices

1. Écris une fonction `uint8_t inverse_bits(uint8_t x)` qui renverse l'ordre des bits (0b10110000 → 0b00001101).
2. Implémente un tampon circulaire (ring buffer) de 32 octets avec `put()` / `get()` — la structure de données de tout driver UART.

3. Écris une FSM de feu tricolore : vert 5 s → orange 1 s → rouge 5 s, avec un « appui piéton » qui écourte le vert.
 4. Décode cette trame de 4 octets `{0xA5, id, valeur_hi, valeur_lo}` en une structure, en vérifiant l'octet de tête.
- ➡ Suite : [Module 02 — C++](#) (les objets sans perdre le contrôle) ou directement [Module 03 — Arduino](#) pour pratiquer sur du vrai matériel.

FORMATION EMBARQUÉE

Module 02 — C++

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 02 — C++ : la puissance des objets, appliquée à l'embarqué

Le C++ est un « C avec des super-pouvoirs » : classes, RAII, templates, bibliothèque standard. Arduino est en réalité du C++. L'enjeu en embarqué : profiter des abstractions **sans coût caché** (le principe « zero-cost abstraction »).

Prérequis : module 01 (tout le C reste valable en C++). Pratique : `g++ main.cpp -o main -Wall -Wextra -std=c++17`.

1. Ce que C++ ajoute immédiatement au C

```
#include <iostream>

int main() {
    int n = 3;
    std::cout << "Valeur : " << n << "\n";    // E/S typées au lieu de printf

    bool ok = true;                            // vrai type booléen
    auto x = 42u;                               // déduction de type (x est unsigned)
    int& ref = n;                               // référence : alias sûr (jamais nul)
    ref = 5;                                    // n vaut 5
}
```

- **Références** (`int&`) : comme un pointeur toujours valide et sans `*`. Passage de gros objets sans copie : `void f(const Mesure& m)`.
- **Surcharge de fonctions** : plusieurs `afficher(int)`, `afficher(float)`.
- **Paramètres par défaut** : `void init(uint32_t baud = 115200);`
- `nullptr` remplace `NULL`, `constexpr` remplace beaucoup de macros :

```
constexpr uint32_t F_CPU = 16'000'000;        // calculé à la compilation
constexpr uint32_t ms_en_ticks(uint32_t ms) { return ms * (F_CPU / 1000); }
static_assert(ms_en_ticks(1) == 16000);       // vérifié à la compilation !
```

2. Classes et objets

Une classe regroupe **données + fonctions** qui les manipulent, avec un contrôle d'accès.

```

class Led {
public:
    Led(uint8_t broche) : broche_(broche), allumee_(false) { // constructeur
        pinMode(broche_, OUTPUT);
    }

    void allumer() { digitalWrite(broche_, HIGH); allumee_ = true; }
    void eteindre() { digitalWrite(broche_, LOW); allumee_ = false; }
    void basculer() { allumee_ ? eteindre() : allumer(); }
    bool estAllumee() const { return allumee_; } // const : ne modifie rien

private:
    uint8_t broche_; // inaccessible de l'extérieur → invariants protégés
    bool allumee_;
};

Led statut(13);
statut.basculer();

```

Concepts clés : - **Encapsulation** : `private` protège l'état interne ; l'extérieur passe par l'interface `public` .
 - **Constructeur** : garantit qu'un objet naît dans un état valide. La **liste d'initialisation** (: `broche_(broche)`) initialise avant le corps. - **Destructeur** `~Led()` : appelé automatiquement à la destruction — la clé du RAII (§3). - `this` : pointeur vers l'objet courant. - **struct vs class** : identiques, sauf visibilité par défaut (public vs private).

3. RAII : l'idée la plus importante du C++

Resource Acquisition Is Initialization : une ressource (mémoire, verrou, périphérique, fichier) est acquise dans le constructeur et **libérée automatiquement** dans le destructeur. Plus d'oubli possible.

```

class SectionCritique {
public:
    SectionCritique() { noInterrupts(); } // acquisition
    ~SectionCritique() { interrupts(); } // libération GARANTIE
};

void copier_compteur(uint32_t& dst, volatile uint32_t& src) {
    SectionCritique sc; // interruptions coupées ici
    dst = src;
} // ← réactivées ici, même si exception/return anticipé

```

En embarqué, RAII sert pour : sections critiques, chip-select SPI, mutex RTOS, transactions I2C. C'est l'argument n°1 pour passer de C à C++.

4. Héritage et polymorphisme

4.1 Héritage

```
class Capteur {                                // classe de base
public:
    virtual ~Capteur() = default;              // destructeur virtuel si héritage !
    virtual int16_t lire() = 0;                 // = 0 → méthode "pure" : classe abstraite
    const char* nom() const { return nom_; }
protected:
    explicit Capteur(const char* nom) : nom_(nom) {}
private:
    const char* nom_;
};

class CapteurTemp : public Capteur {
public:
    CapteurTemp() : Capteur("temp") {}
    int16_t lire() override {                  // override : le compilateur vérifie
        return lire_adc() / 4;
    }
};
```

4.2 Polymorphisme dynamique

```
Capteur* capteurs[] = { &temp, &pression, &humidite };
for (Capteur* c : capteurs) {
    log(c->nom(), c->lire());    // la BONNE méthode est appelée pour chaque type
}
```

Sous le capot : une **vtable** (table de pointeurs de fonctions) — coût modeste (une indirection + un pointeur par objet) mais réel. En embarqué on l'utilise volontiers pour les **drivers interchangeables** (même interface, matériels différents), et on l'évite dans les chemins ultra-critiques.

4.3 Alternative sans coût : le polymorphisme statique (templates)

```
template <typename Bus>
class EcranOled {
public:
    explicit EcranOled(Bus& bus) : bus_(bus) {}
    void pixel(uint8_t x, uint8_t y) { bus_.ecrire(/*...*/); }
private:
    Bus& bus_;
};

EcranOled<BusI2c> ecran(i2c);    // résolu à la COMPILATION : zéro indirection
```

5. Templates et bibliothèque standard

5.1 Fonctions et classes génériques

```
template <typename T>
constexpr T borner(T v, T mini, T maxi) {
    return v < mini ? mini : (v > maxi ? maxi : v);
}
borner<int16_t>(t, -400, 850);

template <typename T, size_t N>
class TamponCirculaire { // taille fixée à la compilation : zéro malloc
public:
    bool put(const T& v) { /* ... */ }
    bool get(T& v) { /* ... */ }
private:
    T buf_[N];
    size_t tete_ = 0, queue_ = 0;
};
TamponCirculaire<uint8_t, 64> rx_uart;
```

5.2 STL : quoi utiliser en embarqué

OK sur micro	À éviter sur petit micro (allocation dynamique)
<code>std::array<T,N></code>	<code>std::vector</code> , <code>std::string</code> , <code>std::map</code>
<code>std::optional</code> , <code>std::pair</code>	<code>std::function</code> (peut allouer)
<code><algorithm></code> (<code>std::min</code> , <code>std::sort</code> , <code>std::clamp</code>)	flux <code>iostream</code> (très lourds)
<code>std::atomic</code> (si supporté)	exceptions et RTTI (souvent désactivés : <code>-fno-exceptions</code> <code>-fno-rtti</code>)

Sur Linux embarqué (Raspberry Pi, i.MX...), toute la STL est utilisable.

6. C++ moderne utile (C++11 → C++17)

```
auto v = lire_capteur(); // déduction de type

for (auto& mesure : historiques) { /* ... */ } // boucle "range-based"

enum class Mode : uint8_t { Repos, Mesure, Erreur }; // enum typée, sans fuite de noms
Mode m = Mode::Mesure;

// Lambdas : fonctions anonymes – parfaites en callback
bouton.surAppui([]() { led.basculer(); });

auto seuil = 100;
capteurs.filtrer([seuil](int16_t v) { return v > seuil; }); // capture de seuil

// Gestion mémoire automatique (Linux embarqué / gros systèmes)
#include <memory>
auto driver = std::make_unique<DriverSpi>(config); // libéré automatiquement
```

Et les mots-clés de robustesse : `override`, `final`, `= delete` (interdire la copie d'un driver : `Led(const Led&) = delete;`), `explicit` (interdire les conversions implicites du constructeur).

7. Différences C / C++ qui piègent

1. C++ est plus strict sur les conversions de pointeurs (`void*` → cast explicite requis).
2. `struct` / `enum` : plus besoin de `typedef`.
3. Pour appeler du C depuis du C++ (drivers constructeur, HAL) :

```
extern "C" {
#include "stm32f1xx_hal.h" // empêche le "name mangling" C++
}
```

1. Les objets globaux ont des **constructeurs exécutés avant** `main()` — attention si le constructeur touche au matériel pas encore initialisé.
2. `new` / `delete` remplacent `malloc` / `free` (et appellent les constructeurs/destructeurs) — mêmes réserves qu'en C sur les petits micros.

8. Exemple complet : bouton anti-rebond + LED, en classes

```
#include <stdint>

class BoutonDebounce {
public:
    explicit BoutonDebounce(uint8_t broche, uint16_t delai_ms = 20)
        : broche_(broche), delai_(delai_ms) {
        pinMode(broche_, INPUT_PULLUP);
    }

    // À appeler à chaque tour de boucle. Renvoie true sur UN appui détecté.
    bool appuye(uint32_t maintenant_ms) {
        bool brut = (digitalRead(broche_) == LOW);
        if (brut != dernier_brut_) {           // changement → on relance le chrono
            t_changement_ = maintenant_ms;
            dernier_brut_ = brut;
        }
        if ((maintenant_ms - t_changement_) > delai_ && brut != etat_stable_) {
            etat_stable_ = brut;               // état confirmé
            return etat_stable_;               // front d'appui uniquement
        }
        return false;
    }

private:
    uint8_t broche_;
    uint16_t delai_;
    bool dernier_brut_ = false;
    bool etat_stable_ = false;
    uint32_t t_changement_ = 0;
};

BoutonDebounce bouton(2);
Led led(13);

void loop() {
    if (bouton.appuye(millis())) led.basculer();
}
```

Remarque : aucun `delay()` — le programme reste réactif. Ce style « non bloquant » est développé au module 03.

9. Quand choisir C ou C++ ?

Situation	Choix
Micro 8 bits minuscule, driver très bas niveau	C
Projet Arduino / ESP32 / STM32 applicatif	C++ (sous-ensemble raisonnable)
Base de code d'équipe, drivers interchangeable	C++ (interfaces, RAII)
Linux embarqué, application riche	C++ complet (ou Rust, module 06)
Contribution au noyau Linux, code hérité	C

Le « C++ embarqué raisonnable » : classes, RAII, templates, `constexpr`, références — mais pas d'exceptions, pas de RTTI, pas d'allocation dynamique imprévisible.

Exercices

1. Réécris le tampon circulaire du module 01 en classe template `TamponCirculaire<T, N>` avec tests.
2. Crée une interface abstraite `IAfficheur` (méthodes `effacer()`, `texte(x, y, str)`) et deux implémentations factices (console, "LCD"). Fais tourner le même code applicatif sur les deux.
3. Écris une classe RAII `ChipSelect` qui abaisse une broche CS dans le constructeur et la relâche dans le destructeur.
4. Transforme la FSM du feu tricolore (module 01) en classe avec la méthode `tick(uint32_t maintenant_ms)`.

➡ Suite : **Module 03 — Arduino** : on branche du vrai matériel.

FORMATION EMBARQUÉE

Module 03 — Arduino

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 03 — Arduino : l'embarqué par la pratique

Arduino = une carte à microcontrôleur + un IDE + un écosystème de bibliothèques. C'est le meilleur terrain d'entraînement : chaque concept des modules 00-02 s'y touche du doigt. Le langage est du **C++** avec deux fonctions imposées : `setup()` et `loop()` .

1. Le matériel

1.1 Cartes courantes

Carte	Micro	Flash/RAM	Tension	Points forts
Uno / Nano	ATmega328P (8 bits, 16 MHz)	32 Ko / 2 Ko	5 V	La référence pour apprendre
Mega 2560	ATmega2560	256 Ko / 8 Ko	5 V	Beaucoup de broches
ESP32	Xtensa 32 bits, 240 MHz	4 Mo / 520 Ko	3,3 V	Wi-Fi + Bluetooth intégrés
ESP8266	32 bits, 80 MHz	4 Mo / 80 Ko	3,3 V	Wi-Fi pas cher
RP2040 (Pico)	2 cœurs ARM M0+	2 Mo / 264 Ko	3,3 V	PIO (E/S programmables)

Commence sur **Uno ou Nano** (robuste, 5 V, documentation infinie), passe à l'**ESP32** dès que tu veux du réseau.

1.2 Broches de l'Uno

- **D0–D13** : numériques (entrée/sortie). D0/D1 = UART (éviter). Les broches marquées  (3, 5, 6, 9, 10, 11) font du **PWM**.
- **A0–A5** : entrées analogiques (ADC 10 bits). A4/A5 = I2C (SDA/SCL).
- **D10–D13** : SPI (CS, MOSI, MISO, SCK).
- Alimentation : 5 V, 3,3 V, GND, Vin (7–12 V).
- ⚠ Max ~20 mA par broche : une LED exige sa **résistance série** (220 Ω) ; un moteur exige un **transistor ou driver** — jamais en direct.

1.3 Outils

- **IDE Arduino 2.x** (simple) ou **VS Code + PlatformIO** (pro : gestion de bibliothèques par projet, debug, multi-cartes). Passe à PlatformIO dès que tu es à l'aise.
- Simulateurs sans matériel : **Wokwi** (<https://wokwi.com>, gratuit, dans le navigateur) et Tinkercad Circuits.

2. Structure d'un sketch

```
// Constantes de câblage en tête de fichier
const uint8_t BROCHE_LED = 13;

void setup() {                                // exécuté UNE fois au démarrage
    pinMode(BROCHE_LED, OUTPUT);
    Serial.begin(115200);                     // ouvre le port série vers le PC
}

void loop() {                                 // exécuté en boucle, indéfiniment
    digitalWrite(BROCHE_LED, HIGH);
    delay(500);                               // bloque 500 ms (on fera mieux, §5)
    digitalWrite(BROCHE_LED, LOW);
    delay(500);
}
```

En coulisses, l'environnement fournit un `main()` qui appelle `setup()` puis `loop()` en boucle infinie — exactement le squelette bare-metal du module 01.

3. Les E/S fondamentales

3.1 Numérique

```
pinMode(7, INPUT_PULLUP);                    // bouton entre broche et GND
bool appuye = (digitalRead(7) == LOW);       // pull-up → LOW = appuyé (logique inversée)

pinMode(13, OUTPUT);
digitalWrite(13, HIGH);
```

3.2 Analogique

```
int brut = analogRead(A0);                   // 0..1023 pour 0..5 V
float volts = brut * (5.0 / 1023.0);
int pct = map(brut, 0, 1023, 0, 100);        // règle de trois intégrée

analogWrite(9, 128);                         // PWM ~50 % (0..255) – pas un vrai DAC !
```

3.3 Port série (ton meilleur ami pour déboguer)

```
Serial.begin(115200);
Serial.print("temp = ");
Serial.println(temperature);

if (Serial.available()) {           // des octets sont arrivés du PC
    char c = Serial.read();
    if (c == '1') digitalWrite(13, HIGH);
}
```

Le **Moniteur série** de l'IDE affiche ces messages ; le **Traceur série** trace les valeurs en courbes.

4. Capteurs et actionneurs classiques

```
// ---- Potentiomètre → luminosité LED
int pot = analogRead(A0);
analogWrite(9, pot / 4);           // 0..1023 → 0..255

// ---- Capteur ultrason HC-SR04 (distance)
digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
long duree = pulseIn(ECHO, HIGH); // µs aller-retour
float cm = duree / 58.0;

// ---- Servo-moteur
#include <Servo.h>
Servo s;
s.attach(9);
s.write(90);                       // angle 0..180°

// ---- Température/humidité DHT22 (bibliothèque "DHT sensor library")
#include <DHT.h>
DHT dht(2, DHT22);
dht.begin();
float t = dht.readTemperature();

// ---- I2C : écran OLED SSD1306 (bibliothèque Adafruit SSD1306)
// ---- SPI : carte SD (bibliothèque SD)
```

Installer une bibliothèque : IDE → Croquis → Inclure une bibliothèque → Gérer les bibliothèques. Avec PlatformIO : `lib_deps` dans `platformio.ini`.

5. LE tournant : programmer sans `delay()`

`delay()` fige tout : impossible de surveiller un bouton pendant qu'une LED clignote. La solution : horodater avec `millis()` (millisecondes depuis le démarrage) et tester le temps écoulé.

```

const uint32_t PERIODE = 500;
uint32_t derniere_bascule = 0;

void loop() {
    uint32_t maintenant = millis();

    if (maintenant - derniere_bascule >= PERIODE) { // soustraction : robuste
        derniere_bascule = maintenant;           // au débordement 49 jours
        digitalWrite(13, !digitalRead(13));
    }

    // ...la boucle reste libre pour lire boutons, capteurs, série...
}

```

Ce motif (« blink without delay ») + la machine d'états du module 01 = tu peux gérer *plusieurs* activités simultanées sur un seul micro sans OS. C'est la compétence Arduino qui sépare débutant et intermédiaire.

6. Interruptions

```

volatile uint32_t compte_tours = 0; // volatile : partagé avec l'ISR !

void isr_capteur() { // ISR : courte, pas de Serial, pas de delay
    compte_tours++;
}

void setup() {
    pinMode(2, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(2), isr_capteur, FALLING);
}

void loop() {
    noInterrupts(); // section critique : lecture atomique
    uint32_t copie = compte_tours;
    interrupts();
    // ...utiliser copie...
}

```

Sur Uno, seules D2 et D3 ont des interruptions externes ; sur ESP32, presque toutes les broches.

7. Projet fil rouge : station météo connectée

Monte en compétence par étapes — chaque étape est un projet fonctionnel :

1. **Blink** puis blink sans `delay()` . (GPIO, millis)
2. **Bouton anti-rebond** qui change le mode de clignotement. (entrées, FSM)
3. **Lecture DHT22** affichée sur le moniteur série toutes les 2 s. (capteur, bibliothèque, timing non bloquant)
4. **Écran OLED I2C** qui affiche température + humidité. (I2C)

5. **Enregistrement sur carte SD** en CSV avec horodatage RTC. (*SPI, fichiers*)
6. **Version ESP32** : envoi des mesures en Wi-Fi (HTTP ou MQTT vers un broker Mosquitto). (*réseau, JSON*)
7. **Alerte** : seuil réglable par potentiomètre, buzzer + LED rouge, hystérésis pour éviter le clignotement autour du seuil. (*ADC, logique*)

À la fin tu auras utilisé : GPIO, ADC, PWM, I2C, SPI, UART, interruptions, FSM, réseau — c'est-à-dire *tout* le module 00 en vrai.

8. Bonnes pratiques et pièges Arduino

1. **int fait 16 bits sur Uno** : `millis()` retourne un `uint32_t`, stocke-le dans un `uint32_t`, pas un `int`.
 2. La RAM fait **2 Ko** : `Serial.println(F("texte"))` garde la chaîne en flash au lieu de la RAM.
 3. `String` (la classe Arduino) fragmente le tas → sur Uno, préférer les tableaux de `char`.
 4. Débranche D0/D1 pendant le téléversement si tu y as branché quelque chose.
 5. Alimente les moteurs/servos par une **alimentation séparée** (masses communes !) — les pics de courant redémarrent la carte.
 6. Structure ton code en fonctions/classes dès 50 lignes ; un sketch n'est pas obligé d'être un plat de spaghettis.
 7. Mesure la place : l'IDE affiche l'usage flash/RAM à chaque compilation.
-

9. Après Arduino : STM32 et le monde pro

Quand Arduino te semblera petit : - **STM32 "Blue Pill" / Nucleo + STM32CubeIDE** : tu configures les horloges et périphériques toi-même (HAL en C), tu débogues avec ST-Link. C'est l'environnement des offres d'emploi « firmware ». - Le chemin : refaire la station météo sur STM32 avec la HAL, puis avec **FreeRTOS** (module 06), puis en accès registres direct. - Tout cela est détaillé pas à pas dans le **Module 10 — STM32**.

Exercices

1. Chenillard de 4 LED, vitesse réglée par potentiomètre, sans `delay()`.
2. Compteur de passages : capteur ultrason + interruption timer, affichage OLED, remise à zéro par bouton (anti-rebond).
3. Thermostat : DHT22 + relais (simulé par LED) + hystérésis de 0,5 °C + consigne réglable, le tout en machine d'états.
4. Sur Wokwi si pas de matériel : reproduis les exercices 1 à 3.

→ Suite : **Module 04 — VHDL & FPGA** : on quitte le logiciel pour décrire du matériel.

FORMATION EMBARQUÉE

Module 04 — VHDL & FPGA

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 04 — VHDL & FPGA : décrire du matériel, pas écrire du logiciel

⚠ Changement de paradigme. Le VHDL **n'est pas un langage de programmation** : c'est un langage de **description de matériel** (HDL). Ton code ne s'« exécute » pas ligne par ligne — il est **synthétisé** en portes logiques et bascules qui fonctionnent toutes **en parallèle, tout le temps**. Oublie la boucle, pense circuit.

1. FPGA : c'est quoi ?

Un **FPGA** (Field-Programmable Gate Array) est une puce contenant des milliers de blocs logiques configurables (LUT + bascules), reliés par un réseau d'interconnexions programmable. On y « charge » un circuit décrit en VHDL ou Verilog.

	Microcontrôleur	FPGA
Exécution	séquentielle (1 instruction à la fois)	parallèle (tout à chaque front d'horloge)
Latence	variable (interruptions, pipeline)	déterministe au cycle près
Fort pour	logique applicative, coût	traitement vidéo/signal, protocoles rapides, temps réel dur
Modifiable	reflasher le programme	recharger le <i>bitstream</i> (le circuit change !)

Outils et matériel

- **Simulation gratuite sans carte** : **GHDL** + **GTKWave** (open source), ou <https://www.edaplayground.com> dans le navigateur. **Commence par là.**
- **Synthèse** : AMD/Xilinx **Vivado** (cartes Basys 3, Arty), Intel/Altera **Quartus** (DE10-Lite), Gowin EDA (Tang Nano 9K à ~15 €), ou le flot open source Yosys/nextpnr (Lattice iCE40/ECP5).
- Flot de travail : écrire → **simuler** (testbench) → synthétiser → placer-router → vérifier le timing → charger le bitstream.

2. Structure d'un fichier VHDL

Tout module a deux parties : l'**entité** (la boîte : nom + broches) et l'**architecture** (ce qu'il y a dedans).

```

library ieee;
use ieee.std_logic_1164.all;      -- types std_logic / std_logic_vector
use ieee.numeric_std.all;        -- unsigned/signed et arithmétique

entity porte_et is
  port (
    a : in  std_logic;           -- entrée 1 bit
    b : in  std_logic;
    s : out std_logic           -- sortie
  );
end entity;

architecture rtl of porte_et is
begin
  s <= a and b;                  -- affectation CONCURRENTE : du fil + une porte
end architecture;

```

Points de syntaxe : - Insensible à la casse ; commentaires avec `--` . - `<=` = affectation de **signal** (≠ `:=` pour les variables). - `rtl` (Register Transfer Level) est un nom d'architecture conventionnel.

2.1 Les types essentiels

```

signal bit_seul : std_logic;      -- '0', '1', 'Z'(haute imp.), '- '...
signal bus8     : std_logic_vector(7 downto 0); -- 8 fils, bit 7 = MSB
signal compteur : unsigned(15 downto 0); -- vecteur qui sait compter
signal temp     : signed(11 downto 0); -- signé (complément à 2)
signal n        : integer range 0 to 99; -- pour compteurs bornés

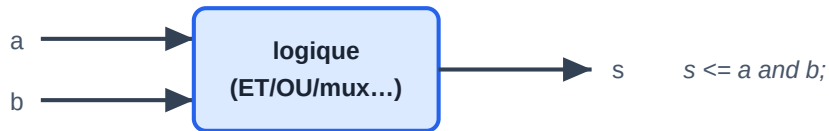
bus8 <= "10100101"; -- littéral binaire
bus8 <= x"A5";      -- littéral hexa
compteur <= compteur + 1; -- OK sur unsigned (pas sur std_logic_vector)
bus8 <= std_logic_vector(compteur(7 downto 0)); -- conversions explicites

```

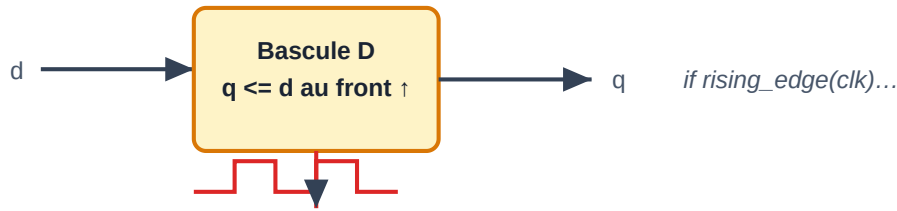
Utilise `unsigned` / `signed` (de `numeric_std`) pour tout ce qui compte, `std_logic_vector` pour les bus « neutres » aux interfaces.

3. Concurrent vs séquentiel : le cœur du VHDL

Combinatoire — la sortie ne dépend QUE des entrées (pas de mémoire)



Séquentiel — la sortie dépend aussi du PASSÉ (registre cadencé)



Logique combinatoire (sans mémoire) contre séquentielle (registre cadencé)

3.1 Instructions concurrentes (hors process)

Chaque ligne décrit un morceau de circuit **permanent et simultané** — l'ordre des lignes ne compte pas :

```
s1 <= a and b;
s2 <= not c;
y <= s1 or s2;                                -- trois "morceaux de circuit" en parallèle

-- Multiplexeur : selon la valeur de sel
with sel select
    y <= a when "00",
        b when "01",
        c when others;

-- Conditionnelle concurrente
y <= a when (en = '1') else '0';
```

3.2 Le process : décrire un bloc en style séquentiel

Un **process** contient du code lu séquentiellement *par le simulateur/ synthétiseur* pour en déduire un circuit.

Deux formes canoniques :

Combinatoire (sensible à toutes ses entrées) :

```
process (a, b, sel)                            -- liste de sensibilité : TOUTES les entrées lues
begin
    if sel = '1' then
        y <= a;
    else
        y <= b;
    end if;
end process;                                   -- chaque signal affecté doit l'être dans
                                              -- TOUS les chemins, sinon → latch involontaire !
```

Séquentiel / synchrone (sensible à l'horloge) — c'est un **registre** :

```

process (clk)
begin
    if rising_edge(clk) then          -- au front montant...
        if reset = '1' then
            q <= (others => '0');    -- reset synchrone
        else
            q <= d;                  -- une bascule D par bit
        end if;
    end if;
end process;

```

3.3 Les deux règles qui évitent 90 % des bugs de débutant

1. **Un signal n'est affecté que dans UN seul process** (sinon : conflit, « multiple drivers »).
2. Dans un process combinatoire, **tout signal de sortie reçoit une valeur dans toutes les branches** (mettre une affectation par défaut en tête du process), sinon le synthétiseur crée un **latch** — presque toujours un bug.

3.4 Signaux vs variables

- **signal** (`<=`) : prend sa nouvelle valeur **à la fin du delta-cycle** — dans un process synchrone, la lecture voit encore l'ancienne valeur. C'est ce qui modélise fidèlement une bascule.
 - **variable** (`:=` , locale au process) : mise à jour **immédiate**. Utile pour du calcul intermédiaire combinatoire dans un process.
-

4. Les briques de base à savoir écrire les yeux fermés

4.1 Compteur avec division d'horloge (faire clignoter une LED)

```
entity blink is
    generic ( F_CLK : natural := 50_000_000 );          -- paramètre : 50 MHz
    port ( clk : in std_logic; led : out std_logic );
end entity;

architecture rtl of blink is
    signal cpt      : unsigned(25 downto 0) := (others => '0');
    signal led_int  : std_logic := '0';
begin
    process (clk)
    begin
        if rising_edge(clk) then
            if cpt = F_CLK/2 - 1 then                    -- toutes les 0,5 s
                cpt      <= (others => '0');
                led_int <= not led_int;
            else
                cpt <= cpt + 1;
            end if;
        end if;
    end process;
    led <= led_int;
end architecture;
```

Note : **pas de `delay()` possible** — attendre = compter des fronts d'horloge.

4.2 Registre à décalage, détecteur de front

```
-- Détecteur de front montant sur une entrée (déjà synchrone)
process (clk)
begin
    if rising_edge(clk) then
        btn_prec <= btn;
    end if;
end process;
front <= btn and not btn_prec;      -- '1' pendant exactement 1 cycle
```

Toute entrée **asynchrone** (bouton, signal externe) doit d'abord passer par **2 bascules en série** (synchroniseur) pour éviter la métastabilité.

4.3 Machine d'états finis (FSM) — le motif central

```
architecture rtl of feu_tricolore is
    type t_etat is (VERT, ORANGE, ROUGE);
    signal etat : t_etat := ROUGE;
    signal cpt : unsigned(27 downto 0) := (others => '0');
begin
    process (clk)
    begin
        if rising_edge(clk) then
            cpt <= cpt + 1;
            case etat is
                when VERT =>
                    if cpt = DUREE_VERT then
                        etat <= ORANGE; cpt <= (others => '0');
                    end if;
                when ORANGE =>
                    if cpt = DUREE_ORANGE then
                        etat <= ROUGE; cpt <= (others => '0');
                    end if;
                when ROUGE =>
                    if cpt = DUREE_ROUGE then
                        etat <= VERT; cpt <= (others => '0');
                    end if;
            end case;
        end if;
    end process;

    -- Sorties décodées de l'état (style Moore : sorties = f(état))
    feu_vert  <= '1' when etat = VERT  else '0';
    feu_orange <= '1' when etat = ORANGE else '0';
    feu_rouge  <= '1' when etat = ROUGE  else '0';
end architecture;
```

Compare avec la FSM en C du module 01 : même concept, mais ici l'« évaluation » a lieu à chaque front d'horloge, physiquement.

5. Hiérarchie : instancier des composants

On assemble des modules comme des puces sur une carte :

```

architecture rtl of top is
    signal tick : std_logic;
begin
    u_div : entity work.diviseur          -- instantiation directe
        generic map ( RATIO => 50_000_000 )
        port map ( clk => clk, tick => tick );

    u_fsm : entity work.feu_tricolore
        port map ( clk => clk, en => tick,
                    feu_vert => led(0), feu_orange => led(1), feu_rouge => led(2) );
end architecture;

```

generic = paramètre de compilation (largeur de bus, fréquence...) → modules réutilisables.

6. Le testbench : simuler AVANT de synthétiser

Règle d'or : **on ne charge jamais sur carte un module non simulé**. Un testbench est un module VHDL sans ports qui instancie le module à tester (DUT), génère horloge et stimuli, et vérifie les sorties.

```

library ieee;
use ieee.std_logic_1164.all;

entity tb_porte_et is end entity;          -- pas de ports

architecture sim of tb_porte_et is
    signal a, b, s : std_logic := '0';
begin
    dut : entity work.porte_et port map (a => a, b => b, s => s);

    stim : process
    begin
        a <= '0'; b <= '0'; wait for 10 ns;
        assert s = '0' report "0 and 0 devrait donner 0" severity error;
        a <= '1'; b <= '0'; wait for 10 ns;
        assert s = '0' severity error;
        a <= '1'; b <= '1'; wait for 10 ns;
        assert s = '1' severity error;
        report "Test termine";
        wait;                                -- stoppe le process
    end process;
end architecture;

```

Avec GHDL :

```

ghdl -a porte_et.vhd tb_porte_et.vhd  # analyse
ghdl -e tb_porte_et                  # élaboration
ghdl -r tb_porte_et --wave=ondes.ghw  # simulation
gtkwave ondes.ghw                    # visualiser les chronogrammes

```

`wait for 10 ns`, boucles de stimulation, `report` ... existent **en simulation seulement** — non synthétisables. Apprendre à lire un **chronogramme** dans GTKWave est une compétence à part entière.

7. Synthétisable ou pas ?

Synthétisable ✓	Simulation seulement ✗
<code>if/case</code> , opérateurs logiques et arithmétiques	<code>wait for 10 ns</code>
process avec <code>rising_edge(clk)</code>	<code>after 5 ns</code> dans les affectations
<code>for ... generate</code> , boucles bornées (déroulées)	fichiers, <code>report</code> (ignoré)
division par puissance de 2	division/modulo quelconques (coûteux/refusés)

Toujours garder en tête : « quel circuit suis-je en train de décrire ? » Si tu ne peux pas le dessiner (portes, mux, registres), le synthétiseur non plus.

8. Notions de timing (aperçu)

Entre deux bascules, le signal traverse de la logique combinatoire : ce chemin doit être plus court qu'une période d'horloge (moins la marge). Les outils font une **analyse statique de timing (STA)** et te disent la fréquence max. Tu declares tes horloges dans un fichier de **contraintes** (`.xdc` chez Xilinx, `.sdc` chez Intel/Altera) qui fixe aussi le brochage :

```
# extrait .xdc (Basys 3)
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.0 [get_ports clk] ;# 100 MHz
```

9. VHDL vs Verilog

Les deux décrivent la même chose. VHDL : verbeux, fortement typé, dominant en Europe (aéronautique, défense, écoles françaises). Verilog/SystemVerilog : concis, dominant aux USA et dans le monde ASIC. Apprends VHDL à fond ; lire du Verilog viendra tout seul ensuite.

Exercices (fais-les tous sur GHDL/EDA Playground)

1. Écris et teste un **multiplexeur 4 vers 1** (2 bits de sélection) — version concurrente et version process.
2. **Compteur BCD 0–9** avec sortie 7 segments (décodeur combinatoire) ; testbench qui vérifie les 10 motifs.
3. **Anti-rebond matériel** : synchroniseur 2 bascules + compteur qui exige 20 ms de stabilité — puis détecteur de front.

4. **Feu tricolore** ci-dessus, enrichi d'un bouton piéton (le vert passe à l'orange au plus tard 1 s après l'appui).
 5. **Émetteur UART 115200 bauds** : FSM start/8 données/stop + compteur de « baud tick ». Vérifie la trame au chronogramme. (Projet sérieux — c'est le « hello world » avancé du FPGA.)
-  Suite : [Module 05 — Java](#) ou [Module 06 — Autres langages](#).

FORMATION EMBARQUÉE

Module 05 — Java

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 05 — Java : la POO complète et son rôle autour de l'embarqué

Java ne tourne pas sur un microcontrôleur 8 bits — il lui faut une machine virtuelle (JVM). Son rôle dans ton parcours : **au-dessus** de l'embarqué — supervision industrielle, applications Android qui pilotent tes objets, serveurs qui reçoivent les données IoT, outillage. Et c'est un excellent professeur de POO, qui consolidera ton C++.

Installation : un JDK (Temurin/OpenJDK 21), et IntelliJ IDEA Community ou VS Code. Compiler/exécuter à la main :

```
javac Main.java # → Main.class (bytecode)
java Main       # exécuté par la JVM
```

1. Java vs C/C++ : ce qui change

	C/C++	Java
Compilation	code machine natif	bytecode exécuté par la JVM (portable)
Mémoire	manuelle (<code>malloc</code> / <code>new</code> + <code>delete</code>)	ramasse-miettes (garbage collector)
Pointeurs	oui, arithmétique incluse	non — références seulement, pas d'accès mémoire brut
Héritage multiple	oui (C++)	non (mais interfaces multiples)
Débordements de tampon	ton problème	exception <code>ArrayIndexOutOfBoundsException</code>
Où ça tourne	partout, y compris nu	là où il y a une JVM (PC, serveur, Android)

Conséquence : impossible d'écrire un driver en Java, mais tu ne chasseras jamais un pointeur fou.

2. Bases du langage

```
public class Main {
    public static void main(String[] args) { // point d'entrée
        int n = 42; // types primitifs : int(32b), long(64b),
        double d = 3.14; // short, byte, float, double, boolean, char
        boolean ok = true;
        String nom = "capteur"; // String = objet immuable

        // Attention embarqué : byte est SIGNÉ (-128..127), pas de unsigned en Java !
        int nonSigné = octet & 0xFF; // le masque classique pour "désigner" un octet

        if (n > 10) System.out.println("grand");
        for (int i = 0; i < 5; i++) { }
        while (ok) { break; }
        switch (n) { case 42 -> System.out.println("réponse"); default -> {} }

        int[] tab = {1, 2, 3};
        for (int v : tab) System.out.println(v); // for-each
    }
}
```

3. POO à la Java

3.1 Classes, encapsulation

```
public class Capteur {
    private final String nom; // final = affecté une fois (comme const)
    private double derniereMesure;

    public Capteur(String nom) { // constructeur
        this.nom = nom;
    }

    public double lire() {
        derniereMesure = acquerir();
        return derniereMesure;
    }

    public String getNom() { return nom; } // accesseur (convention JavaBeans)

    private double acquerir() { return Math.random() * 100; }
}
```

Une classe publique par fichier, fichier du même nom (`Capteur.java`). Tout objet se crée avec `new` et vit sur le tas — le GC le libère quand plus personne ne le référence.

3.2 Héritage, interfaces, polymorphisme

```
public interface Mesurable {                                // interface = contrat pur
    double lire();
    default String unite() { return "u"; } // méthode par défaut possible
}

public class CapteurTemperature extends Capteur implements Mesurable {
    public CapteurTemperature() { super("temp"); }

    @Override
    public double lire() { return super.lire() / 4.0; }

    @Override
    public String unite() { return "°C"; }
}

List<Mesurable> capteurs = List.of(new CapteurTemperature(), new CapteurPression());
for (Mesurable c : capteurs)
    System.out.println(c.lire() + " " + c.unite()); // polymorphisme
```

En Java **toutes les méthodes sont virtuelles** par défaut (contrairement à C++). Les interfaces remplacent l'héritage multiple : une classe en implémente autant qu'elle veut. Autres briques : classes `abstract`, `record` (classe de données immuable en une ligne : `record Mesure(String capteur, double valeur, long ts) {}`), `enum` riches (avec champs et méthodes).

4. Exceptions et robustesse

```
try {
    int v = Integer.parseInt(texte);
} catch (NumberFormatException e) {
    System.err.println("Entrée invalide : " + e.getMessage());
} finally {
    // toujours exécuté (nettoyage)
}

// try-with-resources = le RAII de Java (ferme automatiquement)
try (var port = new Socket("192.168.1.50", 502)) {
    // ... parler Modbus TCP à un automate ...
} // socket fermé ici, même en cas d'exception
```

Les exceptions **vérifiées** (`IOException` ...) doivent être attrapées ou déclarées (`throws IOException`) — le compilateur y veille.

5. Collections et génériques

```
List<Double> mesures = new ArrayList<>(); // tableau dynamique
mesures.add(21.5);

Map<String, Capteur> parNom = new HashMap<>(); // dictionnaire
parNom.put("temp", capteurTemp);

Deque<Byte> fifo = new ArrayDeque<>(); // file (ton ring buffer, en mieux)

// Streams : traiter des données de capteurs en une passe déclarative
double moyenne = mesures.stream()
    .filter(v -> v > 0)
    .mapToDouble(Double::doubleValue)
    .average()
    .orElse(0.0);
```

Les génériques (`List<Double>`) = les templates de C++, en plus simple (mais effacés à l'exécution).

6. Threads : la concurrence sans registre

```
Thread lecteur = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {
        double v = capteur.lire();
        file.offer(v); // file thread-safe
        try { Thread.sleep(1000); } catch (InterruptedException e) { return; }
    }
});
lecteur.start();
```

Outils clés : `synchronized` (section critique), `AtomicInteger`, `BlockingQueue` (producteur/consommateur — le pattern des passerelles IoT), `ExecutorService` (pool de threads). Les concepts (course critique, section critique, atomicité) sont **exactement ceux des ISR** du module 00 — Java te les fait pratiquer confortablement.

7. Java et le monde embarqué/industriel : cas concrets

7.1 Parler à un automate en Modbus TCP

Avec une bibliothèque comme *digitalpetri/modbus* ou *j2mod* :

```
// Pseudo-code représentatif (API j2mod simplifiée)
ModbusTCPMaster maitre = new ModbusTCPMaster("192.168.1.50");
maitre.connect();
Register[] regs = maitre.readMultipleRegisters(0, 10); // 10 mots à l'adresse 0
int temperature = regs[0].getValue();
maitre.writeSingleRegister(100, new SimpleRegister(1)); // commander une sortie
```

C'est le pont typique entre tes modules 07/08 (automates) et un logiciel de supervision maison.

7.2 Recevoir des mesures IoT en MQTT (Eclipse Paho)

```
MqttClient client = new MqttClient("tcp://broker:1883", "superviseur");
client.connect();
client.subscribe("usine/four1/temperature", (topic, msg) -> {
    double t = Double.parseDouble(new String(msg.getPayload()));
    if (t > 80) alerter(t);
});
```

L'ESP32 du module 03 publie ; ton serveur Java s'abonne, historise (JDBC → PostgreSQL), alerte, affiche.

7.3 Où Java apparaît dans l'industrie

- **Android** : applis qui pilotent tes cartes en Bluetooth/Wi-Fi (Kotlin, cousin direct de Java).
- **Supervision/SCADA** : Ignition (plateforme SCADA très répandue) est en Java ; ses scripts sont en Jython.
- **Serveurs & passerelles** : Spring Boot pour exposer les données machines en API REST ; Eclipse Kura pour les passerelles IoT.
- **Outils** : beaucoup d'outils EDA/IDE (dont Eclipse, la base de STM32CubeIDE et de TIA-like) sont écrits en Java.

8. Mini-projet : superviseur de station météo

Complète le projet fil rouge du module 03 :

1. L'ESP32 publie `{"t":21.5,"h":48}` sur le topic MQTT `meteo/salon`.
2. Application Java : s'abonne, déserialise le JSON (bibliothèque Jackson), stocke en base SQLite (JDBC), calcule min/max/moyenne glissante.
3. Expose `GET /mesures/dernieres` en HTTP (un simple `com.sun.net.httpserver.HttpServer` suffit pour commencer).
4. Bonus : seuil d'alerte configurable + envoi d'un e-mail.

Tu auras construit une **chaîne IoT complète** : capteur → firmware C++ → réseau → back-end Java.

Exercices

1. Modélise `Capteur` (abstraite), `CapteurTemp`, `CapteurHum`, et une classe `Station` qui interroge une `List<Capteur>` et journalise en CSV.
2. Implémente producteur/consommateur : un thread « acquisition » (simulé), un thread « traitement » avec `BlockingQueue`, arrêt propre par interruption.
3. Écris un décodeur de trame : `byte[] {0xA5, id, hi, lo}` → objet `Mesure` (attention au signe des `byte` : masque `& 0xFF` !).
4. Parseur de log : lis un fichier CSV de mesures, sors min/max/moyenne par capteur avec les streams.

→ Suite : [Module 06 — Autres langages utiles.](#)

FORMATION EMBARQUÉE

Module 06 — Autres langages

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 06 — Les autres langages et outils indispensables de l'embarqué

Autour du trio C/C++/VHDL gravitent des outils qu'on croise dans tout projet réel : l'assembleur (pour comprendre), MicroPython (pour prototyper), Rust (l'avenir proche), Make/CMake (pour construire), un RTOS (pour structurer), Linux embarqué et le shell (pour les grosses cibles).

1. Assembleur : lire plus qu'écrire

On n'écrit presque plus d'assembleur, mais **savoir le lire** est irremplaçable : déboguer un crash, optimiser une routine, comprendre ce que le compilateur fait vraiment.

1.1 Le modèle : registres + instructions

Un CPU ARM Cortex-M a 16 registres (R0–R12, SP pile, LR retour, PC compteur de programme). Une instruction = une opération élémentaire :

```
; ARM Thumb – équivalent de : c = a + b;
LDR  r0, =var_a      ; r0 ← adresse de a
LDR  r1, [r0]         ; r1 ← contenu à cette adresse (charge a)
LDR  r0, =var_b
LDR  r2, [r0]
ADD  r3, r1, r2       ; r3 ← r1 + r2
LDR  r0, =var_c
STR  r3, [r0]         ; mémorise le résultat dans c
```

Architecture **load/store** : on charge en registre, on calcule, on range.

1.2 Ce qu'il faut savoir faire

- Reconnaître : **MOV**, **LDR/STR**, **ADD/SUB**, **CMP** + branchements (**BEQ**, **BNE**, **B**), **PUSH/POP**, **BL** (appel de fonction).
- Comprendre la **convention d'appel** ARM : arguments dans R0–R3, retour dans R0 — c'est ce qui rend C et assembleur interopérables.
- Générer et lire l'assembleur de ton propre C :

```
arm-none-eabi-gcc -S -Os main.c -o main.s    # voir ce que produit le compilateur
arm-none-eabi-objdump -d firmware.elf       # désassembler un binaire
```

Exercice royal : compiler une fonction simple avec **-O0** puis **-O2** et comparer — tu ne verras plus jamais l'optimiseur de la même façon.

2. MicroPython / CircuitPython : prototyper en minutes

Python interprété *sur* le microcontrôleur (ESP32, Raspberry Pi Pico, cartes PyBoard). Idéal pour valider un capteur ou une idée avant de coder en C.

```
# Raspberry Pi Pico – blink + capteur, en 10 lignes
from machine import Pin, ADC, I2C
import time

led = Pin(25, Pin.OUT)
pot = ADC(26)

while True:
    led.toggle()
    print("pot =", pot.read_u16())
    time.sleep(0.5)
```

- **REPL** sur le port série : tu tapes du code *en direct* sur la carte.
- Outils : Thonny (IDE débutant), `mpremote`, VS Code.
- Limites : 10–100× plus lent que le C, RAM consommée par l'interpréteur, latences non déterministes → pas pour le temps réel dur.
- Python côté PC est de toute façon indispensable : scripts d'analyse de logs, génération de tables, bancs de test (pyserial pour parler à tes cartes), traitement des mesures (matplotlib, pandas).

3. Rust embarqué : la sécurité mémoire sans ramasse-miettes

Rust apporte les garanties de Java (pas de pointeur fou, pas de course de données) **sans machine virtuelle ni GC** — vérifié à la compilation par le *borrow checker*. Adopté par Linux (noyau), l'automobile et l'industrie.

```
// Blink sur STM32 avec le HAL embassy/stm32 (représentatif)
#![no_std]                // pas de bibliothèque standard : bare-metal
#![no_main]

use embassy_stm32::gpio::{Level, Output, Speed};
use embassy_time::Timer;

#[embassy_executor::main]
async fn main(_spawner: embassy_executor::Spawner) {
    let p = embassy_stm32::init(Default::default());
    let mut led = Output::new(p.PC13, Level::High, Speed::Low);

    loop {
        led.toggle();
        Timer::after_millis(500).await; // async SANS OS : tâches coopératives
    }
}
```

Concepts propres à Rust : **ownership** (chaque valeur a un unique propriétaire), **borrowing** (`&` , `&mut` vérifiés), `Result<T, E>` au lieu d'exceptions, `unsafe` pour les rares accès registres bruts (isolés et audités). Apprends-le **après** le C : tu comprendras *pourquoi* chaque règle existe. Point d'entrée : *The Rust Book* puis *The Embedded Rust Book* (gratuits en ligne).

4. Make, CMake et la ligne de commande

4.1 Makefile : l'automatisation historique

```
CC      = arm-none-eabi-gcc
CFLAGS  = -mcpu=cortex-m3 -mthumb -Os -Wall -Wextra
SRCS    = main.c uart.c capteur.c
OBJS    = $(SRCS:.c=.o)

firmware.elf: $(OBJS)
    $(CC) $(CFLAGS) $^ -T stm32.ld -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

flash: firmware.elf
    openocd -f interface/stlink.cfg -f target/stm32f1x.cfg \
        -c "program firmware.elf verify reset exit"

clean:
    rm -f $(OBJS) firmware.elf
```

Une règle = `cible: dépendances` + commandes (indentées par une **tabulation**). Make ne reconstruit que ce qui a changé.

4.2 CMake : le standard moderne

```
cmake_minimum_required(VERSION 3.20)
project(firmware C)
add_executable(firmware main.c uart.c capteur.c)
target_compile_options(firmware PRIVATE -Wall -Wextra -Os)
```

CMake *génère* des Makefiles (ou Ninja). C'est l'outil des projets STM32Cube, ESP-IDF, Zephyr, Pico SDK — incontournable en entreprise.

4.3 Le shell et Git

Le quotidien de l'embarqué se passe dans un terminal :

```
ls, cd, cat, grep -r "motif" src/, find . -name "*.c"
picocom -b 115200 /dev/ttyUSB0          # console série (ou minicom, screen)
dmesg | tail                            # "quel port série ma carte a-t-elle pris ?"
hexdump -C firmware.bin | head          # inspecter un binaire

git init / add / commit / push          # versionne TOUT, dès le premier projet
git diff / log / branch / merge
```

5. FreeRTOS : structurer le firmware en tâches

Quand la super-boucle + FSM ne suffit plus (plusieurs activités à échéances différentes), un **RTOS** ordonnance des tâches par priorité, de façon préemptive et déterministe. FreeRTOS est le plus répandu (et intégré à l'ESP32 d'office).

```
void tache_capteur(void *param) {
    for (;;) {
        mesure_t m = lire_capteur();
        xQueueSend(file_mesures, &m, portMAX_DELAY); // vers l'autre tâche
        vTaskDelay(pdMS_TO_TICKS(100));              // dort 100 ms (le CPU est libre)
    }
}

void tache_affichage(void *param) {
    mesure_t m;
    for (;;) {
        if (xQueueReceive(file_mesures, &m, portMAX_DELAY) == pdTRUE)
            afficher(m);
    }
}

int main(void) {
    file_mesures = xQueueCreate(16, sizeof(mesure_t));
    xTaskCreate(tache_capteur, "capteur", 256, NULL, 2, NULL);
    xTaskCreate(tache_affichage, "affich", 512, NULL, 1, NULL);
    vTaskStartScheduler(); // ne revient jamais
}
```

Concepts à maîtriser : tâche, priorité, préemption, **file (queue)**, **sémaphore**, **mutex** (et l'inversion de priorité), taille de pile par tâche. Alternatives : **Zephyr** (le RTOS qui monte, style Linux) et l'approche async de Rust/embassy.

6. Linux embarqué (aperçu)

Dès que la cible est costaud (Raspberry Pi, passerelles, box), on embarque un vrai Linux :

- **Espace utilisateur** : tes programmes C/C++/Python accèdent au matériel via `/dev` (`/dev/ttyUSB0` , `/dev/i2c-1` , `/dev/spidev0.0` , `gpiod`).
- **Device Tree** : description du matériel fournie au noyau.

- **Yocto / Buildroot** : usines à images Linux sur mesure.
- **Drivers noyau** : modules en C — le sommet de la programmation bas niveau.

C'est un domaine d'emploi énorme (« BSP engineer », « Linux embedded »). Y aller après avoir bien pratiqué C + shell.

7. Quel langage pour quel travail ? (synthèse du cours)

Besoin	Outil
Firmware micro, driver, RTOS	C (puis Rust)
Application embarquée riche (ESP32, STM32, Linux)	C++
Prototype rapide, banc de test, scripts	Python / MicroPython
Logique matérielle, FPGA	VHDL (ou Verilog)
Supervision, Android, back-end IoT	Java (ou Kotlin/C#)
Automates industriels	IEC 61131-3 (modules 07-08)
Comprendre / déboguer au plus bas	Assembleur (lecture)
Construire, flasher, automatiser	Make/CMake + shell + Git

Exercices

1. Compile une fonction C (`-O0` vs `-O2`), désassemble, et explique deux optimisations que tu observes.
2. Écris un Makefile pour un projet C à 3 fichiers avec cibles `all` , `clean` , et reconstruction minimale.
3. Sur ESP32 (Arduino + FreeRTOS intégré) : deux tâches — l'une clignote une LED à 2 Hz, l'autre imprime un compteur chaque seconde — communiquant par une queue.
4. En MicroPython sur Pico (ou simulateur Wokwi) : refais la station météo du module 03 et compare le temps de développement avec la version C++.

➡ Suite : [Module 07 — Suite Siemens](#) : on entre dans l'industrie.

FORMATION EMBARQUÉE

Module 07 — Suite Siemens

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

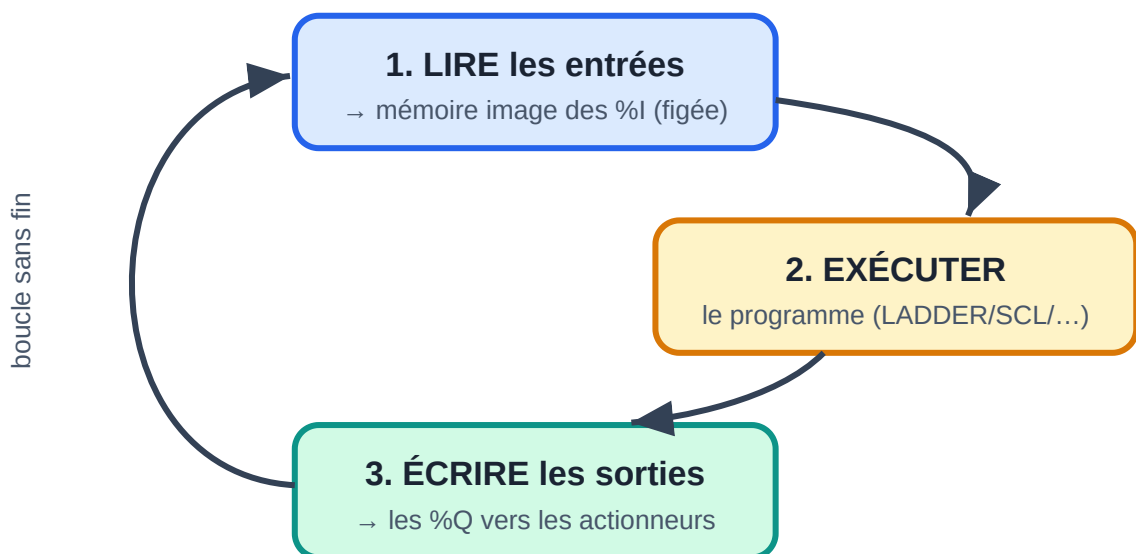
Module 07 — Suite Siemens : TIA Portal et les automates S7

Les **automates programmables (PLC)** sont les ordinateurs durcis qui pilotent usines, machines, lignes de production. Siemens (gamme **SIMATIC**) est le n°1 mondial. Son environnement unique s'appelle **TIA Portal** (Totally Integrated Automation) : programmation, configuration matérielle, écrans opérateurs et réseau au même endroit.

1. Le monde de l'automatisme en 10 points

1. Un PLC exécute un **cycle** permanent : *lire les entrées* → *exécuter le programme* → *écrire les sorties* (typiquement 1–10 ms par tour). Retiens : les entrées sont figées en **mémoire image** pendant tout le cycle.

Le cycle automate (scan) — 1 à 10 ms, en boucle permanente



Le cycle automate (scan) en trois temps

1. Entrées typiques : capteurs TOR (tout-ou-rien : fins de course, cellules), analogiques (4–20 mA, 0–10 V), codeurs.
2. Sorties typiques : contacteurs, électrovannes, variateurs de vitesse, voyants.
3. Les langages sont normalisés **IEC 61131-3** : LADDER (LD), blocs fonctionnels (FBD), texte structuré (ST), liste d'instructions (IL, obsolète), SFC/GRAFCET.
4. La sûreté prime : arrêts d'urgence câblés ou en **PLC de sécurité** (S7 F-CPU), modes manuel/auto, gestion des défauts.

5. Réseaux : **PROFINET** (Ethernet industriel Siemens), PROFIBUS (historique), IO-Link (capteurs), OPC UA (vers l'informatique).
6. **HMI/SCADA** : les écrans opérateurs (Siemens : WinCC).
7. Le matériel se « câble » aussi dans le logiciel : la **configuration matérielle** (racks, modules, adresses) fait partie du projet.
8. Vocabulaire d'atelier : consigne, retour (feedback), verrouillage (interlock), front, temporisation, cadence, cycle machine.
9. On ne redémarre pas une usine « pour voir » : simulation (**PLCSIM**) et procédures de mise en service sont essentielles.

2. Le matériel SIMATIC

Gamme	Usage	Notes
LOGO!	micro-automatisme (portail, éclairage)	programmation en blocs, très simple
S7-1200	machines petites/moyennes	LA gamme pour apprendre ; CPU 1214C classique
S7-1500	lignes de production, process	performances, diagnostic, sécurité, motion
ET 200SP	E/S déportées sur PROFINET	s'ajoutent aux CPU ci-dessus
S7-300/400	anciennes générations	encore partout en usine (maintenance !)

Adressage des E/S (exemples) : - **%I0.0** : entrée TOR bit 0 de l'octet 0 — **%I** = Input. - **%Q0.3** : sortie TOR — **%Q** = Output. - **%IW64** : mot d'entrée analogique (16 bits). - **%M10.0** : memento (bit interne) ; **%MW20** : mot interne. - **"MonDB".vitesse** : variable dans un bloc de données (voir §4).

3. TIA Portal : prise en main

- Versions : TIA Portal V17/V18/V19... Chaque CPU exige une version compatible.
- **Licence d'essai 21 jours** téléchargeable chez Siemens ; l'université ou l'employeur ont souvent des licences. Le simulateur **PLCSIM** permet de tout tester **sans automate**.
- Les deux vues : *Portal view* (guidée) et *Project view* (l'arbre du projet — celle qu'on utilise vraiment).

Créer un premier projet : 1. « Créer un projet » → « Configurer un appareil » → choisir la CPU exacte (ex. CPU 1214C DC/DC/DC, référence 6ES7 214-...). 2. La **vue des appareils** montre le rack : on y règle adresses IP (PROFINET), propriétés, et on ajoute les modules d'E/S. 3. **Table des variables API (tags)** : nommer les E/S — jamais d'adresses brutes dans le code :

Nom	Type	Adresse
bp_marche	Bool	%I0.0
bp_arret	Bool	%I0.1
capteur_haut	Bool	%I0.2
moteur	Bool	%Q0.0

1. Programmer dans `Main [OB1]` (voir §4), compiler, puis « Démarrer la simulation » (PLCSIM) ou « Charger dans l'appareil ».
2. **Table de visualisation (watch table)** : lire/forcer des variables en ligne — l'outil de mise au point n°1.

4. L'organisation d'un programme S7

Le programme est fait de **blocs** :

- **OB (Organization Block)** : appelés par le système.
- `OB1` : le cycle principal (ton `loop()`).
- `OB100` : au démarrage (ton `setup()`).
- OB cycliques (`OB30+` : toutes les X ms), OB d'erreur (`OB82` ...).
- **FC (Function)** : fonction **sans mémoire** — que des paramètres d'entrée/sortie et des variables temporaires.
- **FB (Function Block)** : fonction **avec mémoire** : chaque appel possède son **DB d'instance** qui conserve ses données entre les cycles. (FB ≈ classe, DB d'instance ≈ objet — le parallèle avec le C++ est exact.)
- **DB (Data Block)** : bloc de données global (recettes, consignes, états).

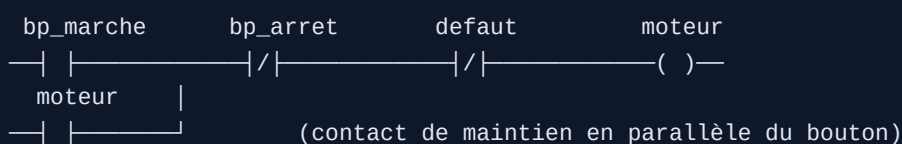
Types de données : `Bool` , `Int` (16 bits), `DInt` (32), `Real` (flottant), `Time` (`T#2s500ms`), `String` , `Array[0..9] of Real` , `Struct` / types UDT.

⚠ S7-1200/1500 : préférer l'**adressage symbolique** et les **DB optimisés** (accès par nom, plus d'offsets manuels).

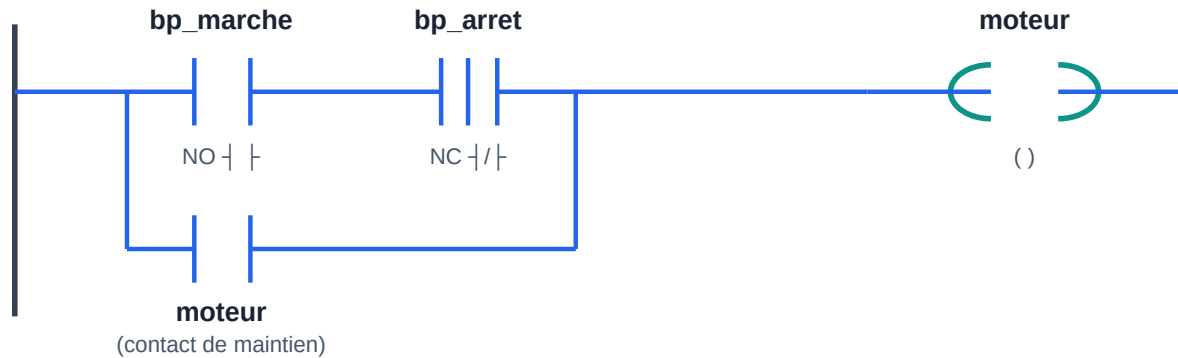
5. LADDER (CONT) : le langage des électriciens

Le LADDER transpose les schémas à relais : des « barreaux » lus de gauche à droite, à chaque cycle, de haut en bas.

5.1 Marche/arrêt avec auto-maintien (LE motif de base)



LADDER — marche/arrêt avec auto-maintien (le motif de base)



Barreau LADDER : marche/arrêt avec auto-maintien

- ---|--- contact NO (vrai si bit à 1) ; ---|/|--- contact NF (vrai si bit à 0).
- $()$ bobine : écrit le résultat dans le bit.
- Lecture : le moteur démarre si **bp_marche** ET PAS **bp_arret** ET PAS **default** ; une fois parti, il **se maintient** par son propre contact.
- Équivalent : bobines **(S)** / **(R)** (Set/Reset) — attention, la dernière écriture du cycle gagne.

5.2 Fronts, temporisations, compteurs

- **Front montant** : ---|P|--- (avec un bit de mémoire associé) — vrai un seul cycle. Indispensable pour compter des pièces, pas des durées d'appui.
- **Temporisations IEC** (des FB à instancier) :
- **TON** : retard à l'enclenchement (Q passe à 1 après PT de IN vrai) ;
- **TOF** : retard au déclenchement ; **TP** : impulsion calibrée.
- **Compteurs** : **CTU** (comptage), **CTD**, **CTUD** avec consigne **PV** et sortie **Q** quand atteinte.

Exemple : tapis roulant qui s'arrête 5 s après le dernier carton détecté → un **TOF** avec **PT := T#5s** sur la cellule.

5.3 Comparaisons et calcul en LADDER

Boîtes **CMP >=**, **ADD**, **MOVE**, **SCALE_X** / **NORM_X** (mise à l'échelle des analogiques : convertir **%IW64** (0..27648) en 0..100 °C).

6. FBD (LOG) : les blocs logiques

Même sémantique que le LADDER, dessinée en portes ET/OU/NON reliées par des fils. Certains préfèrent FBD pour la logique combinatoire dense ; les Allemands l'utilisent beaucoup. Sache lire les deux — le contenu (contacts, temporisations, FB) est identique.

7. SCL : le texte structuré (ton avantage de développeur)

SCL (Structured Control Language) est le ST de l'IEC 61131-3, proche du Pascal. Avec ton bagage C, c'est ton langage naturel pour tout ce qui est calcul, boucles, chaînes, algorithmes.

```

// FB "RegulationNiveau" – corps en SCL
IF #niveau_mesure < #consigne - #hysteresis THEN
    #pompe := TRUE;
ELSIF #niveau_mesure > #consigne + #hysteresis THEN
    #pompe := FALSE;
END_IF;

// Mise à l'échelle d'une entrée analogique 0..27648 → 0..10 m
#niveau_mesure := INT_TO_REAL(#brut) / 27648.0 * 10.0;

// Boucle sur un tableau
FOR #i := 0 TO 9 DO
    #somme := #somme + #mesures[#i];
END_FOR;
#moyenne := #somme / 10.0;

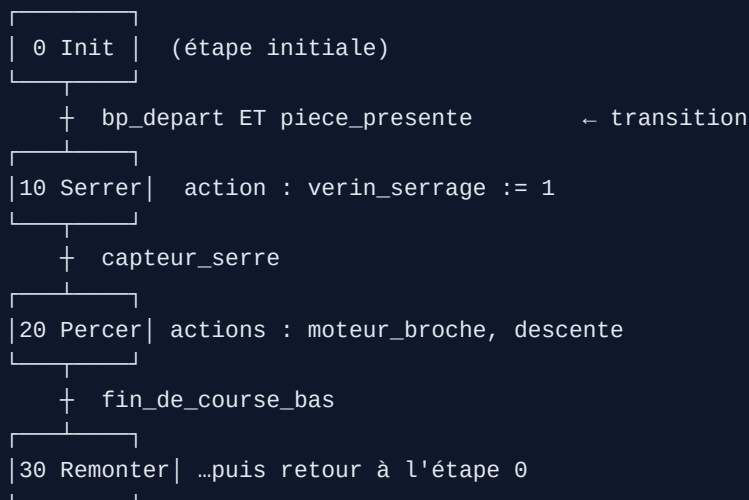
CASE #etape OF
    0: IF #bp_depart THEN #etape := 10; END_IF;
    10: #verin_sortir := TRUE;
        IF #capteur_sorti THEN #etape := 20; END_IF;
    20: // ...
END_CASE;

```

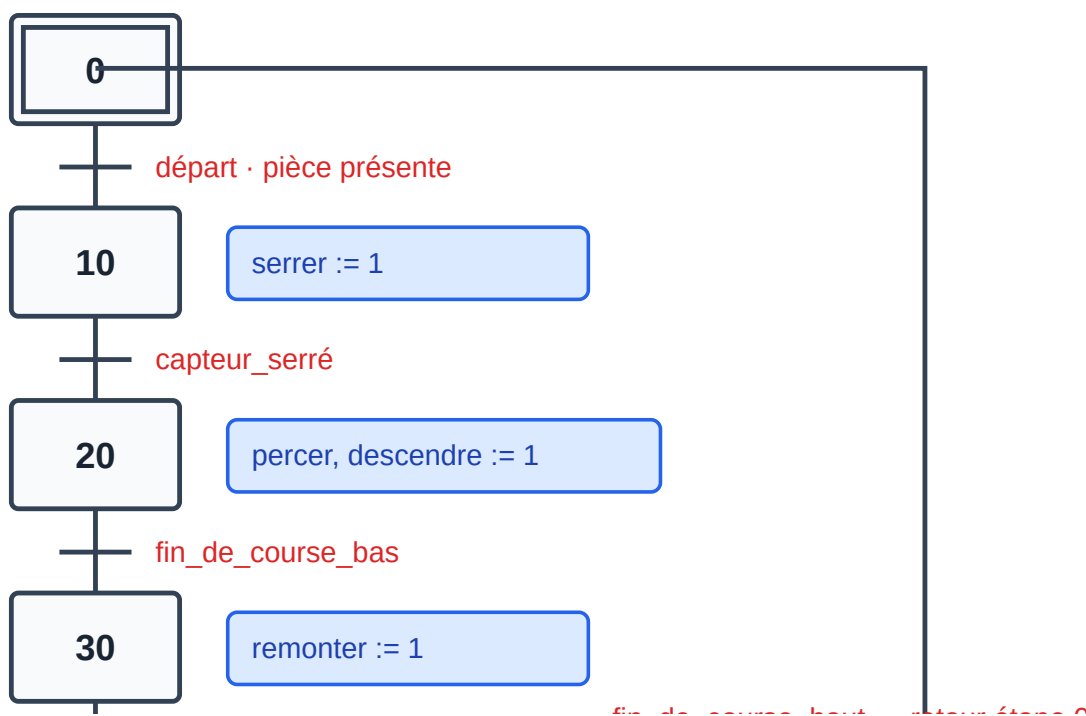
Syntaxe : `:=` affectation, `=` comparaison, `#var` variable locale du bloc, `"MonDB".x` accès global, mots-clés `AND OR NOT XOR`, blocs terminés par `END_IF/END_FOR/END_CASE`. Bonnes pratiques d'usine : LADDER pour les verrouillages et la logique visible par la maintenance, **SCL pour les calculs et les séquences complexes**.

8. GRAFCET / SFC : les séquences

Le **GRAFCET** (norme française devenue IEC 60848 / SFC) décrit un cycle machine en **étapes** (actions) séparées par des **transitions** (conditions) :



GRAFCET — étapes (actions) séparées par des transitions (conditions)



Structure d'un GRAFCET : étapes, actions et transitions

Chez Siemens, l'éditeur graphique s'appelle **S7-GRAPH** (option, S7-1500). Sans lui, on implémente le GRAFCET en SCL avec un `CASE #etape OF ...` (voir §7) ou en LADDER avec un bit par étape — sachant dessiner le GRAFCET sur papier d'abord : c'est **l'outil de conception**, exigé dans toute étude d'automatisme française.

9. HMI : WinCC

Dans TIA Portal, on ajoute un pupitre (Basic Panel KTP700, Comfort Panel, ou **WinCC Unified**) au projet :

1. Créer les **vues** (écrans) : boutons, voyants, champs numériques, courbes, jauges.
2. Relier chaque objet à une **variable HMI** pointant vers une variable API (la liaison PROFINET est déjà dans le projet).
3. **Alarmes** : messages déclenchés sur bits/valeurs, avec accusé.
4. **Archives** : historiser des mesures, tracer des courbes.
5. Gestion des **utilisateurs** (opérateur/régleur/admin) par niveaux.
6. Simulation du pupitre sur PC intégrée (avec PLCSIM à côté : machine virtuelle complète sans un seul câble).

10. PROFINET, diagnostic et outils du quotidien

- **PROFINET** : Ethernet industriel temps réel. Chaque appareil a un nom d'appareil + IP ; l'échange d'E/S est cyclique et surveillé. Configuration dans la « vue de réseau » de TIA.
- **Diagnostic** : LED de la CPU, « en ligne et diagnostic », tampon de diagnostic (l'historique des défauts — premier réflexe en dépannage).

- **Comparaison en ligne/hors ligne** : voir si le programme dans l'automate diffère du projet.
- **Forçage** : imposer une E/S pour tester — avec les précautions d'usage.
- **TIA Openness / bibliothèques** : automatiser et standardiser les projets.

11. Parcours d'apprentissage Siemens

1. Installe TIA Portal (essai) + PLCSIM. Sans PC assez costaud : commence par un simulateur LADDER générique, mais vise TIA.
2. Refais en LADDER : marche/arrêt auto-maintenu → télérupteur (front + bascule) → chenillard avec TON → comptage de pièces CTU.
3. Feu tricolore en GRAFCET papier, puis en SCL (**CASE**), puis testé PLCSIM.
4. Mini-projet « station de remplissage » : convoyeur, détection bouteille, remplissage temporisé, compteur de lot, défauts (bourrage = cellule activée trop longtemps), écran WinCC avec marche/arrêt/consigne/alarme.
5. Passe les tutoriels officiels SCE (Siemens Automation Cooperates with Education) : gratuits, en français, excellents.
6. Certification utile en France : les titres pro / BTS CRSA-CIRA s'appuient massivement sur TIA Portal.

Exercices

1. LADDER : va-et-vient d'un chariot entre deux fins de course, avec arrêt d'urgence prioritaire et réarmement obligatoire.
2. SCL : bloc FB **Moyenne_Glissante** (Array de 10 Real, entrée **nouvelle_mesure**, sortie **moyenne**) — puis instancie-le deux fois (deux capteurs).
3. GRAFCET : porte de garage (impulsion → ouvre ; obstacles → réouverture ; fin de course haut/bas ; temporisation de fermeture auto 30 s). Dessine, puis code en **CASE** .
4. HMI : écran avec bouton MARCHE, voyant état moteur, champ consigne vitesse, et une alarme « défaut thermique ».

➔ Suite : **Module 08 — Suite Schneider** : mêmes concepts, autre écosystème.

FORMATION EMBARQUÉE

Module 08 — Suite Schneider

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 08 — Suite Schneider Electric : EcoStruxure, Modicon et Modbus

Schneider Electric est le grand concurrent français de Siemens — très présent en France (bâtiment, eau, énergie, machines). Bonne nouvelle : **90 % des concepts du module 07 se transposent tels quels** (cycle automate, IEC 61131-3, GRAFCET, HMI). Ce module se concentre sur ce qui change : les logiciels, le matériel Modicon, et **Modbus**, le protocole historique inventé par Modicon.

Prérequis : module 07 (les concepts d'automatisme n'y sont pas répétés).

1. La galaxie logicielle Schneider (s'y retrouver dans les noms)

Schneider a renommé toute son offre sous la marque **EcoStruxure** :

Logiciel actuel	Ancien nom	Pour quels automates	Usage
EcoStruxure Machine Expert	SoMachine	M241, M251, M262, M258	machines OEM
EcoStruxure Machine Expert – Basic	SoMachine Basic	M221	petites machines — gratuit , idéal débutant
EcoStruxure Control Expert	Unity Pro	M340, M580 , Premium, Quantum	process, infrastructures
EcoStruxure Machine Expert HVAC / autres	—	M171/M172	CVC
Vijeo Designer / EcoStruxure Operator Terminal Expert	—	écrans Harmony (ex-Magelis)	HMI
EcoStruxure Machine SCADA Expert / AVEVA	—	supervision	SCADA

À noter : **Machine Expert est basé sur CODESYS**, la plateforme IEC 61131-3 utilisée par des dizaines de marques (Wago, Festo, Beckhoff-like...) — le maîtriser te sert donc bien au-delà de Schneider.

Par où commencer sans budget : - *Machine Expert Basic* : téléchargement gratuit, **simulateur intégré** — parfait pour débiter (cible M221). - *Control Expert* : version d'essai ; son **simulateur PLC** remplace l'automate. - *CODESYS* (codesys.com) : gratuit avec soft-PLC de démo — pour pratiquer l'IEC 61131-3 pur.

2. Le matériel Modicon

Gamme	Positionnement	Équivalent Siemens approx.
M221	micro-automate machines simples	S7-1200 entrée de gamme / LOGO!+
M241 / M251 / M262	machines, motion, IIoT (M262)	S7-1200/1500
M340	process moyen	S7-300 / S7-1500
M580 (ePAC)	process & infrastructures, redondance, cybersécurité	S7-1500 / 400H
Premium / Quantum	anciennes générations, encore en service	S7-400
ATV (Altivar)	variateurs de vitesse	SINAMICS
Harmony (ex-Magelis)	écrans opérateurs	pupitres SIMATIC HMI
Lexium	servo/motion	SIMOTICS/SINAMICS S
Zelio Logic	relais programmable	LOGO!

Adressage mémoire (différent de Siemens — à connaître)

Notation IEC « % » avec localisation :

- `%I0.3` : entrée TOR 3 ; `%Q0.2` : sortie TOR 2 (M221 : `%I0.x` / `%Q0.x`).
- `%M10` : bit interne 10 ; `%MW100` : **mot interne** 100 (16 bits) ; `%MD100` : double mot ; `%MF100` : flottant.
- `%IW1.0` : mot d'entrée analogique.
- `%S` / `%SW` : bits/mots **système** (ex. `%S6` = clignotant 1 s sur M221).
- Sur Control Expert / Machine Expert on privilégie les **variables symboliques** (comme chez Siemens) ; les `%MW` restent centraux pour les échanges **Modbus** (voir §5).

3. Programmer : les 5 langages IEC 61131-3, version Schneider

Les langages sont les mêmes qu'au module 07 — seuls l'éditeur et quelques blocs changent.

3.1 LADDER (LD)

Le marche/arrêt auto-maintenu s'écrit à l'identique. Blocs standard :

- Temporisateurs : `TON` , `TOF` , `TP` (IEC) — sur M221 : blocs `%Tm` en mode TON/TOF/TP avec pré-réglage `%Tm.P` .
- Compteurs : `CTU/CTD/CTUD` — sur M221 : `%Ci` .
- Fronts : contacts `P` et `N` directement dans le barreau.
- Comparaison/opération : blocs de comparaison `[%MW10 > 100]` et d'opération `[%MW20 := %MW10 * 2]` insérables dans les barreaux (très pratique, typiquement Schneider).

3.2 Texte structuré (ST)

Identique au SCL de Siemens (c'est la même norme) :

```

(* Régulation de niveau, version Schneider ST *)
IF niveau < consigne - hysteresis THEN
    pompe := TRUE;
ELSIF niveau > consigne + hysteresis THEN
    pompe := FALSE;
END_IF;

(* Accès direct possible aux mots mémoire *)
%MW200 := REAL_TO_INT(debit * 10.0);

FOR i := 0 TO 9 DO
    somme := somme + mesures[i];
END_FOR;

```

3.3 FBD, IL, et SFC/GRAFCET

- **FBD** : identique dans l'esprit.
- **SFC** : Control Expert dispose d'un **véritable éditeur GRAFCET** (héritage Telemecanique — c'est historiquement l'automate « français ») : étapes, transitions, divergences ET/OU, actions qualifiées (N, S, R, L, D). Sur M221 (Machine Expert Basic), pas d'éditeur SFC : on code le GRAFCET en LADDER (un bit par étape) ou avec les bits d'étape **%Xi** selon gamme.
- **DFB** (Derived Function Block, Control Expert) = le FB Siemens : bloc utilisateur avec variables internes, instanciable — même logique « classe/objet ».

3.4 Structure d'un projet

- **Machine Expert (Basic)** : tâche maître cyclique (**MAST**), sections/POU organisées ; configuration matérielle par onglets (modules, E/S, réseau).
- **Control Expert** : tâches **MAST** (cyclique ou périodique), **FAST** (rapide), événementielles ; sections par langage ; variables globales typées ; DFB ; écrans d'exploitation intégrés (mini-HMI de mise au point).
- Tables d'animation = watch tables Siemens (visualiser/forcer).

4. HMI Harmony et Vijeo Designer

Même démarche que WinCC : 1. Choisir le pupitre (Harmony ST6, GTU...), créer les **panneaux** (écrans). 2. Variables liées à l'automate (Modbus TCP en général — souvent adressées directement en **%MW**). 3. Objets : boutons, voyants, bargraphes, courbes, tableaux d'alarmes, recettes, multi-langue, niveaux d'utilisateurs. 4. Simulation sur PC, puis transfert par Ethernet/USB.

Pour les gros systèmes : AVEVA (ex-Wonderware, partenaire de Schneider) — InTouch/System Platform — côté SCADA.

5. Modbus : LE protocole à maîtriser (né chez Modicon en 1979)

Simple, ouvert, omniprésent — des variateurs aux compteurs d'énergie. Deux transports :

- **Modbus RTU** : trame binaire sur **RS-485** (2 fils, multipoint, jusqu'à ~1200 m). Maître unique qui interroge des esclaves (adresse 1–247).

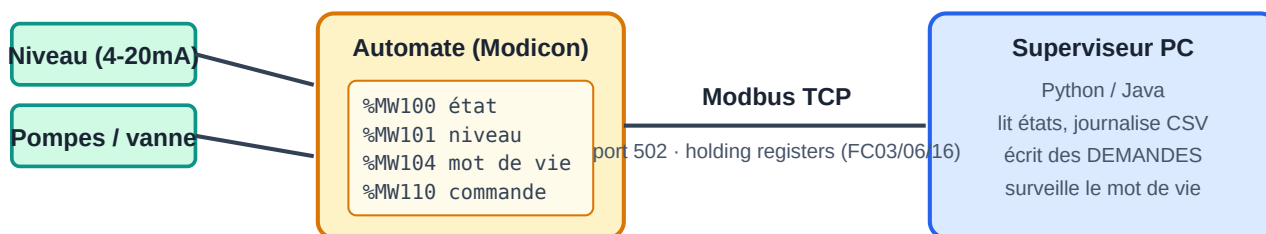
- **Modbus TCP** : la même chose encapsulée dans TCP/IP port **502**.

5.1 Le modèle de données

Zone	Type	Fonctions usuelles
Coils	bits lecture/écriture	01 (lire), 05/15 (écrire)
Discrete inputs	bits lecture seule	02
Holding registers	mots 16 bits R/W	03 (lire), 06/16 (écrire)
Input registers	mots 16 bits lecture	04

Sur un Modicon, les holding registers correspondent naturellement aux **%MW** : « lire 10 registres à l'adresse 100 » = lire **%MW100..%MW109**. C'est pour ça que les échanges HMI/SCADA/supervision Java (module 05 §7.1) se câblent si vite sur du Schneider.

Chaîne automate ↔ supervision via Modbus TCP



chaque mot : Modbus ne transporte que des entiers 16 bits.

demandes ; l'automate valide, borne et décide seul des sorties.

Chaîne automate ↔ supervision via Modbus TCP

5.2 Pièges classiques

- Décalage d'adresse « +1 » (adresse protocole 0 = registre « 40001 » des vieilles documentations).
- Ordre des mots pour les 32 bits (deux registres : lequel en premier ? dépend de l'équipement — « word swap »).
- RS-485 : polarisation et résistances de terminaison, vitesse/parité identiques partout.
- Un seul maître en RTU ; en TCP, attention au nombre de connexions.

5.3 À côté de Modbus

- **CANopen** : historique machines Schneider (M241/M251 en maître).
- **EtherNet/IP** et **OPC UA** : M262/M580 vers l'IT et l'IIoT.
- **IO-Link** : capteurs intelligents.

6. Siemens vs Schneider : synthèse pour ta culture (et tes entretiens)

	Siemens	Schneider
IDE	TIA Portal	Machine Expert / Control Expert
CPU phare	S7-1200 / S7-1500	M221-M262 / M580
Réseau maison	PROFINET / PROFIBUS	Modbus TCP / Ethernet-IP / CANopen
HMI	WinCC	Vijeo / Harmony
ST s'appelle	SCL	ST
FB + données	FB + DB d'instance	DFB (Control Expert) / FB CODESYS
GRAFCET	S7-GRAPH (option)	SFC natif Control Expert
Simulateur	PLCSIM	simulateur intégré / soft-PLC CODESYS
Forces terrain	industrie manufacturière, Allemagne/monde	bâtiment, eau, énergie, France

Sur le marché de l'emploi français en automatisme, exiger « TIA Portal **ou** Unity/Control Expert » est la norme — qui a compris l'un apprend l'autre en quelques semaines. Ce qui compte : IEC 61131-3, GRAFCET, Modbus, lecture de schémas électriques, méthode de mise en service.

7. Parcours d'apprentissage Schneider

1. Installe **Machine Expert Basic** (gratuit) → cible M221 simulée.
2. Refais tous les exercices LADDER du module 07 (auto-maintien, télérupteur, chenillard **%TM**, comptage **%C**).
3. Ajoute la dimension Modbus : depuis un PC, lis/écrit les **%MW** du simulateur avec un client Modbus gratuit (QModMaster, mbpoll) — puis depuis un script Python (**pymodbus**) ou ton programme Java (module 05).
4. Essaie **Control Expert** (essai) : reprends le feu tricolore en **SFC natif**, crée un **DFB** de moyenne glissante, joue avec les tables d'animation.
5. Mini-projet : gestion de cuve (remplissage/vidange, 2 pompes alternées pour user les deux également, sécurité niveau très haut câblée, alarmes, écran Harmony simulé, supervision des **%MW** en Modbus TCP).
6. Ressources : documentation Schneider (excellente, souvent en français), chaînes YouTube d'automatisme francophones, forums (Automation Sense, PLCtalk).

Exercices

1. M221 simulé : démarrage étoile/triangle d'un moteur (2 contacteurs, temporisation 5 s, verrouillage mutuel impératif).
2. ST : bloc **Alternance_Pompes** — à chaque demande, démarre la pompe la moins utilisée (compteurs d'heures en **%MD**), avec report si défaut.

3. Modbus : table d'échange documentée (10 mots : états, mesures, commandes, mot de vie) + lecture depuis `mbpoll` ou `pymodbus` .
 4. GRAFCET SFC : cycle de lavage (prélavage 2 min → lavage 5 min → rinçage ×2 → essorage), pause/reprise, abandon avec vidange sécurisée.
-  Termine par [Module 09 — Parcours & ressources](#).

FORMATION EMBARQUÉE

Module 09 — Parcours & ressources

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 09 — Parcours d'apprentissage et ressources

Un plan réaliste sur 12 mois (à ton rythme : compte les mois en « mois de pratique régulière », ~5-8 h/semaine), les projets jalons, le matériel, et les meilleures ressources gratuites.

1. Le plan sur 12 mois

Phase 1 — Fondations (mois 1-3)

Quoi	Objectif de sortie
Module 00 (fondamentaux)	convertir binaire/hexa de tête, expliquer UART vs I2C vs SPI
Module 01 (C) sur PC	pointeurs, bit à bit, FSM et ring buffer écrits sans aide
Module 03 (Arduino) en parallèle	station météo étapes 1-4 fonctionnelle
Shell + Git (module 06 §4)	chaque projet versionné sur GitHub

Jalon : le thermostat à hystérésis (exercice 3 du module 03) marche, le code est propre, sur GitHub, avec un README.

Phase 2 — Approfondissement (mois 4-6)

Quoi	Objectif de sortie
Module 02 (C++)	station météo refactorée en classes ; RAII compris
Module 03 suite (ESP32)	mesures publiées en MQTT sur le réseau
Module 04 (VHDL) sur simulateur	compteur, FSM, testbenchs GHDL maîtrisés
Module 06 : Make/CMake	un projet compilé sans IDE

Jalon : l'émetteur UART en VHDL (exercice 5 du module 04) simulé et vérifié au chronogramme.

Phase 3 — Industrialisation (mois 7-9)

Quoi	Objectif de sortie
Module 07 (Siemens) avec PLCSIM	station de remplissage + écran WinCC
Module 08 (Schneider) avec Machine Expert Basic	mêmes exercices, plus Modbus
GRAFCET papier	dessiner le GRAFCET AVANT de coder, systématiquement
Module 05 (Java)	superviseur MQTT → SQLite → HTTP du module 05

Jalon : chaîne complète capteur ESP32 → MQTT → Java → base de données, ET un projet automate simulé avec HMI.

Phase 4 — Spécialisation (mois 10-12)

Choisis UNE voie et creuse (c'est ce qui te rend embauchable) :

- **Firmware** : STM32 + HAL + FreeRTOS (module 10 en entier, appuyé sur le module 06 §5), debug SWD, puis refaire un driver en accès registres.
- **FPGA** : carte réelle (Tang Nano/Basys 3), VGA ou filtre audio, timing.
- **Automatisme** : Control Expert + un projet complet avec SFC natif, variateur simulé, supervision — et candidature en alternance/stage.
- **Linux embarqué** : Raspberry Pi sans bureau, GPIO en C via gpod, service systemd, puis Buildroot.
- **Rust embarqué** : The Rust Book puis refaire la station météo en embassy.

Jalon final : un portfolio GitHub avec 3-4 projets documentés (photos, schémas, README) — ça vaut plus qu'un CV en entretien.

2. Budget matériel récapitulatif

Priorité	Matériel	Prix approx.
1	Arduino Uno/Nano + kit démarrage (breadboard, LED, boutons, capteurs, servo)	25-40 €
2	ESP32 DevKit	8-12 €
3	Modules I2C/SPI : OLED SSD1306, DHT22, HC-SR04, module SD, RTC	15-25 €
4	Multimètre correct	20-30 €
5	STM32 Nucleo-64 (ST-Link intégré)	15-20 €
6	FPGA : Tang Nano 9K (15 €) ou Basys 3 (~150 €, si études)	15-150 €
Bonus	Analyseur logique 8 voies (clone Saleae, +sigrok/PulseView) — magique pour VOIR l'UART/I2C/SPI du module 00	10-15 €

Automates : rien à acheter — simulateurs PLCSIM et Machine Expert Basic. (Un M221 ou S7-1200 d'occasion ~100-200 € seulement si tu veux du réel.)

3. Ressources gratuites recommandées

C / C++

- *Modern C* (Jens Gustedt) — PDF gratuit ; en français : cours C d'OpenClassrooms/Zeste de Savoir.
- learncpp.com — LA référence C++ progressive.
- godbolt.org (Compiler Explorer) — voir l'assembleur généré en direct.
- exercism.org (pistes C et C++) — exercices corrigés.

Arduino / microcontrôleurs

- docs.arduino.cc + le forum français.
- wokwi.com — simulateur en ligne (Uno, ESP32, Pico).
- Chaînes YouTube : GreatScott!, Andreas Spiess (EN) ; U=RI, Tuto Arduino (FR).
- deepbluembedded.com et controllerstech.com — tutoriels STM32.

VHDL / FPGA

- ghdl + GTKWave (open source) ; edaplayground.com (navigateur).
- nandland.com — tutoriels VHDL débutant excellents.
- fpga4student.com, hdlbits (Verilog mais concepts identiques).
- Livre : *Free Range VHDL* (gratuit, PDF).

Java

- dev.java (tutoriels officiels Oracle) ; *Java pour les nuls* si besoin FR.
- baeldung.com — référence pratique.
- JetBrains Academy (parcours gratuit partiel).

Automatisme Siemens / Schneider

- **SCE Siemens** (Siemens Automation Cooperates with Education) : modules de formation TIA Portal gratuits, traduits en français.
- Schneider : Machine Expert Basic gratuit + eLearning Schneider (Université).
- realpars.com (payant mais extraits YouTube gratuits très bons), plcacademy.com.
- FR : chaînes YouTube d'enseignants en BTS CRSA/Électrotechnique ; cours de GRAFCET des académies (PDF librement accessibles).
- Norme à connaître de nom : IEC 61131-3 (langages), IEC 60848 (GRAFCET).

Bas niveau / OS / RTOS

- *The Embedded Rust Book*, *The Rust Book* (gratuits).
- freertos.org — documentation et livre PDF gratuit (*Mastering the FreeRTOS Kernel*).
- bootlin.com — supports de formation Linux embarqué en libre accès (société française).
- *Operating Systems: Three Easy Pieces* — OS expliqués, gratuit.

Pratique et communauté

- github.com — publie tout ; lis le code des bibliothèques que tu utilises.
- electronics.stackexchange.com, forum arduino.cc, r/embedded, r/PLC.
- Concours/défis : Advent of Code (algo en C), hackaday.io (projets).

4. Les erreurs de débutant à éviter

1. **Regarder des tutos sans taper le code.** Ratio sain : 1 h de vidéo → 3 h de pratique.

2. Sauter le C pour aller « direct au concret » Arduino : tu plafonneras vite sans pointeurs ni bits.
 3. Traiter le VHDL comme un langage de programmation (boucles, delays...) : pense circuit, simule tout.
 4. Copier des sketchs entiers sans les relire : recopie à la main, modifie, casse, répare.
 5. Négliger l'électronique de base : la moitié des « bugs logiciels » des débutants sont des masses non reliées, des pull-ups absentes ou des alimentations faiblardes.
 6. Apprendre 6 langages en parallèle : suis les phases, une brique à la fois.
 7. Ne pas versionner : Git dès le premier jour, même pour un blink.
 8. En automatisme : coder avant d'avoir dessiné le GRAFCET et listé les E/S.
-

5. Et pour l'emploi (France) ?

Métiers accessibles avec ce parcours : - **Technicien / ingénieur automaticien** (modules 00, 07, 08 + électrotechnique) — forte demande, partout en France. - **Développeur firmware / systèmes embarqués** (00-03, 06 ; C/C++, STM32, RTOS) — aéronautique, automobile, IoT, médical. - **Ingénieur FPGA** (00, 04 ; VHDL, timing) — défense, spatial, télécoms (souvent bac+5, mais le portfolio parle). - **Développeur IoT full-stack** (03, 05, 06 ; du capteur au cloud). - **Test & validation / banc de test** (C, Python, instruments) — excellente porte d'entrée.

Dans tous les cas : le **portfolio GitHub de projets finis et documentés** est ton meilleur argument, surtout en reconversion.

Bon courage — et souviens-toi : *tout* ce cours se résume à une seule habitude : **construire des petits projets qui marchent, souvent.**

FORMATION EMBARQUÉE

Module 10 — STM32

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 10 — STM32 : le microcontrôleur professionnel

Le **STM32** (STMicroelectronics, cœurs ARM Cortex-M) est LE microcontrôleur des offres d'emploi firmware en Europe. C'est la marche entre Arduino et le métier : ici, c'est **toi** qui configures les horloges, les périphériques et les interruptions. Tout ce que les modules 00-02 t'ont appris (registres, pointeurs, **volatile**, ISR) devient quotidien.

Prérequis : modules 00, 01 (indispensables), 02 et 03 (fortement conseillés).

1. La famille STM32 et le matériel pour apprendre

1.1 S'y retrouver dans les références

STM32 F4 11 RE se décode : famille **F4** (performance), sous-famille **11**, boîtier/flash **RE** (64 broches, 512 Ko).

Série	Cœur	Positionnement
F0 / G0	Cortex-M0/M0+	entrée de gamme, remplaçants des 8 bits
F1	Cortex-M3, 72 MHz	l'historique (la « Blue Pill » F103)
F4	Cortex-M4 + FPU, 100-180 MHz	LE standard d'apprentissage et d'industrie
G4	Cortex-M4	contrôle moteur, analogique riche
F7 / H7	Cortex-M7, jusqu'à 550 MHz	haut de gamme (écran, DSP, Ethernet)
L0 / L4 / U5	Cortex-M0+/M4/M33	basse consommation (piles, IoT)
WB / WL	M4 + radio	Bluetooth LE / LoRa intégrés

1.2 Quelle carte acheter ?

Carte	Prix	Avis
Nucleo-F411RE (ou F446RE)	~15-20 €	Le meilleur choix : ST-Link (programmeur/débogueur) intégré, broches Arduino, alim USB
« Blue Pill » STM32F103C8	~3-5 €	Très bon marché, mais exige un ST-Link externe (~5 €) et il circule des clones défectueux
Discovery (F407, avec capteurs)	~25 €	Bien aussi, plus de périphériques embarqués

Recommandation : **une Nucleo**, rien d'autre à acheter — le débogueur est sur la carte.

1.3 Ce qui change par rapport à l'Arduino Uno

	Uno (ATmega328P)	Nucleo-F411
CPU	8 bits, 16 MHz	32 bits ARM, 100 MHz + FPU
Flash / RAM	32 Ko / 2 Ko	512 Ko / 128 Ko
Tension	5 V	3,3 V (⚠ broches non tolérantes 5 V sauf indication)
Timers	3	11, bien plus riches
Débogage	<code>Serial.print</code>	points d'arrêt, pas-à-pas, inspection en direct (SWD)
Config	cachée par l'IDE	horloges, broches, périphériques : à toi

2. Les outils : STM32CubeIDE et CubeMX

- **STM32CubeIDE** (gratuit, Windows/Linux/Mac) : IDE Eclipse + compilateur `arm-none-eabi-gcc` + débogueur intégré + **CubeMX** incorporé.
- **CubeMX** : le configurateur graphique — tu cliques les broches et les périphériques, il **génère le code d'initialisation** en C.
- **HAL** (Hardware Abstraction Layer) : la bibliothèque C de ST (`HAL_GPIO_WritePin(...)`) — portable entre STM32, un peu verbeuse.
- **LL** (Low Layer) : API fine proche des registres, plus rapide.
- Alternatives pro : VS Code + extension STM32, PlatformIO, ou Makefile pur (CubeMX sait générer un projet Makefile).

Créer le premier projet (blink)

1. `File → New → STM32 Project` → chercher « Nucleo-F411RE » (onglet Board Selector) → initialiser les périphériques par défaut : **oui**.
2. L'onglet **Pinout** s'ouvre : la LED verte est déjà sur **PA5** (nom **LD2**), le bouton bleu sur **PC13** .
3. `Project → Generate Code` (ou Alt+K).
4. Dans `main.c` , entre les balises **USER CODE** (⚠ tout code hors de ces balises est **écrasé** à la prochaine génération) :

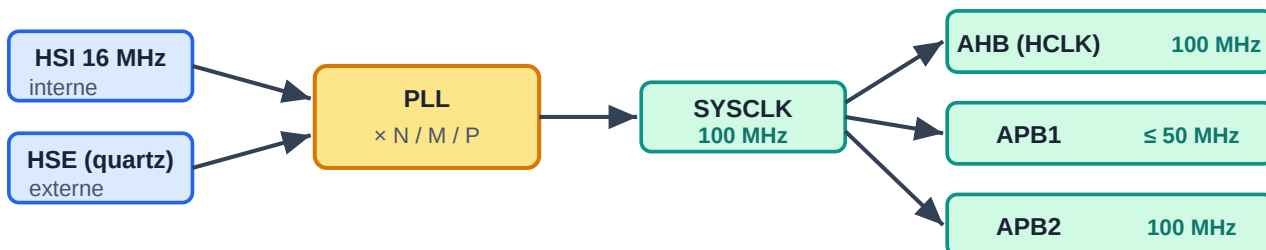
```
/* USER CODE BEGIN WHILE */
while (1) {
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
    HAL_Delay(500); // ms - s'appuie sur le timer SysTick
}
/* USER CODE END WHILE */
```

1. Clic sur le scarabée (Debug) : compilation, flash par ST-Link, arrêt sur `main` . **F8** pour lancer. Pose un point d'arrêt, regarde les variables en direct, ouvre la vue *SFRs* pour voir les **registres** du module 00 en vrai.

3. L'arbre d'horloges (clock tree) — le passage obligé

Sur Arduino, l'horloge est réglée pour toi. Sur STM32, l'onglet **Clock Configuration** de CubeMX montre l'arbre : source (HSI interne 16 MHz ou HSE quartz externe) → **PLL** (multiplie) → **SYSCLK** → prescalers AHB/APB1/APB2 qui cadencent bus et périphériques.

Arbre d'horloges STM32 — d'où vient la fréquence de chaque bloc



→ Chaque périphérique (GPIO, UART, timer...) est cadencé par SON bus. La fréquence d'un timer dépend de l'APB auquel il est accroché : à connaître pour calculer les prescalers (PSC/ARR).

△ Bug n°1 : oublier d'activer l'horloge du périphérique (RCC) → il « ne répond pas ».

Arbre d'horloges STM32

À savoir : - Tape la fréquence cible (ex. 100 MHz) dans la case HCLK : CubeMX résout la PLL tout seul. - Chaque périphérique doit avoir son **horloge activée** (RCC) avant tout accès — CubeMX le fait dans `HAL_...MspInit()` ; en accès registre c'est TON premier réflexe (`RCC->AHB1ENR |= ...`), et l'oublier est LE bug classique n°1 (le périphérique « ne répond pas »). - La fréquence des timers dépend du bus APB auquel ils sont accrochés : toujours vérifier dans le clock tree.

4. Les périphériques essentiels avec la HAL

4.1 GPIO

```
// Configuré dans CubeMX (mode Output/Input/Alternate/Analog, pull-up...)
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) { /* bouton appuyé */ }
```

Interruption externe (**EXTI**) : dans CubeMX, mettre la broche en mode `GPIO_EXTI`, activer la ligne dans l'onglet NVIC, puis :

```
// Callback appelé par la HAL depuis l'ISR — les règles du module 00 s'appliquent
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin) {
    if (GPIO_Pin == B1_Pin) { drapeau_bouton = 1; } // court : un drapeau volatile
}
```

4.2 UART (le Serial de STM32)

Sur Nucleo, l'**USART2** (PA2/PA3) passe par le ST-Link et apparaît comme un port série USB sur le PC — ton canal de debug gratuit.

```
char msg[] = "Bonjour STM32\r\n";
HAL_UART_Transmit(&huart2, (uint8_t*)msg, sizeof msg - 1, 100); // bloquant, timeout 100 ms

uint8_t rx;
HAL_UART_Receive_IT(&huart2, &rx, 1); // version par interruption
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *h) {
    traiter(rx);
    HAL_UART_Receive_IT(&huart2, &rx, 1); // réarmer !
}
```

Astuce confort : rediriger `printf` vers l'UART en implémentant `int __io_putchar(int ch)` — cherché automatiquement par la libc de ST.

4.3 Timers et PWM

Exemple : PWM 1 kHz sur TIM2 canal 1 (PA0). Le timer compte à `f_bus / (PSC+1)` et déborde à `ARR+1` ticks ; le canal compare avec `CCR`.

```
f_PWM = f_timer / ((PSC+1) * (ARR+1))
100 MHz / ((99+1) * (999+1)) = 1 kHz → PSC=99, ARR=999
```

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 250); // rapport cyclique 25 % (250/1000)
```

Interruption périodique (base de temps) : mode « Update event », puis

```
HAL_TIM_Base_Start_IT(&htim3);
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *h) {
    if (h->Instance == TIM3) { tick_1ms(); }
}
```

Autres modes qui font la réputation des timers STM32 : *Input Capture* (mesurer une largeur d'impulsion — ton HC-SR04 sans `pulseIn`), *Encoder Mode* (lire un codeur en quadrature en matériel, zéro CPU), one-pulse, timers synchronisés.

4.4 ADC

12 bits (0..4095), plusieurs canaux scannés en séquence, déclenchable par timer.

```
HAL_ADC_Start(&hadc1);
HAL_ADC_PollForConversion(&hadc1, 10);
uint32_t brut = HAL_ADC_GetValue(&hadc1); // 0..4095
float volts = brut * 3.3f / 4095.0f;
```

4.5 I2C et SPI

```
// I2C : lire 6 octets chez un capteur (adresse 7 bits décalée d'un cran !)  
HAL_I2C_Mem_Read(&hi2c1, 0x68 << 1, 0x3B, 1, buf, 6, 100);  
  
// SPI avec chip select manuel  
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_RESET);  
HAL_SPI_TransmitReceive(&hspi1, tx, rx, len, 100);  
HAL_GPIO_WritePin(CS_GPIO_Port, CS_Pin, GPIO_PIN_SET);
```

⚠ Piège I2C n°1 : la HAL attend l'adresse **déjà décalée à gauche** (`0x68 << 1`), contrairement à la bibliothèque Wire d'Arduino.

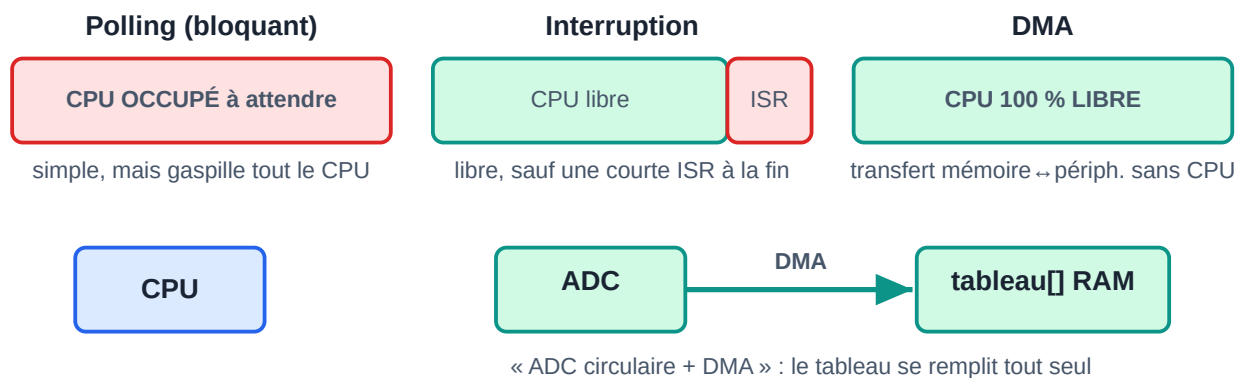
4.6 Les trois modes d'E/S : polling, interruption, DMA

Presque chaque fonction HAL existe en trois versions — c'est un concept central :

Mode	Exemple	CPU pendant le transfert
Bloquant	<code>HAL_UART_Transmit</code>	occupé à attendre
Interruption	<code>HAL_UART_Transmit_IT</code>	libre, ISR à la fin
DMA	<code>HAL_UART_Transmit_DMA</code>	libre, zéro ISR intermédiaire

Le **DMA** (Direct Memory Access) copie mémoire ↔ périphérique sans le CPU : indispensable pour l'ADC continu, l'audio, les gros transferts SPI/UART. Motif star : **ADC en mode circulaire + DMA** — le tableau `mesures[]` se remplit tout seul en tâche de fond, le programme se contente de le lire.

Les 3 modes de transfert — qui occupe le CPU ?



Polling, interruption, DMA : qui occupe le CPU ?

4.7 NVIC : les priorités d'interruption

Le contrôleur d'interruptions ARM (**NVIC**) donne une **priorité réglable à chaque IRQ** (0 = la plus urgente) avec préemption imbriquée — bien plus riche que l'AVR. Dans CubeMX : onglet NVIC. Règle pratique : les E/S rapides (UART, DMA) plus prioritaires que les timers lents ; et si tu utilises FreeRTOS, seules les

priorités numériquement $\geq$ `configMAX_SYSCALL_INTERRUPT_PRIORITY` ont le droit d'appeler l'API RTOS (`...FromISR`) — l'erreur classique de mise en route.

5. Descendre encore : la HAL vue de dessous

Pour comprendre (et pour les entretiens), refais le blink **en accès registres direct**, sans HAL :

```
#include "stm32f4xx.h"           // définitions CMSIS : RCC, GPIOA, structures...

int main(void) {
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;    // 1) horloge du port A – TOUJOURS d'abord
    GPIOA->MODER &= ~GPIO_MODER_MODER5;     // 2) PA5 : nettoyer les 2 bits de mode
    GPIOA->MODER |=  GPIO_MODER_MODER5_0;    // 01 = sortie

    while (1) {
        GPIOA->ODR ^= GPIO_ODR_OD5;         // 3) basculer la sortie
        for (volatile uint32_t i = 0; i < 400000; i++); // attente grossière
    }
}
```

Tout le module 01 est là : pointeurs vers structures de registres (`GPIOA->` est un pointeur vers une `struct` mappée à `0x40020000`), masques, `volatile`. La documentation de référence : le **Reference Manual** (RM0383 pour le F411) — apprendre à y naviguer (chapitre RCC, chapitre GPIO, tableau des registres) est une compétence professionnelle en soi. Le **datasheet** donne le brochage et l'électrique ; le RM donne les registres.

6. Le débogage sérieux (l'énorme avantage du STM32)

- **Points d'arrêt / pas-à-pas / variables live** via SWD — plus jamais de debug à l'aveugle.
 - **Live Expressions** : suivre une variable en continu pendant l'exécution.
 - Vue **SFRs** : lire/écrire les registres périphériques en direct.
 - **Fault Analyzer** : quand le programme part en `HardFault` (pointeur fou, division par zéro), il montre l'instruction et les registres au moment du crash — le module 01 §6.4 en pratique.
 - En dehors de l'IDE : **OpenOCD + GDB** (`arm-none-eabi-gdb`) — le duo des chaînes CI et de Linux.
 - Réflexe pro : à un plantage mystérieux, vérifier 1) horloge du périphérique activée, 2) taille de pile, 3) `volatile` manquant, 4) priorités NVIC vs FreeRTOS.
-

7. FreeRTOS sur STM32

CubeMX intègre FreeRTOS (via la couche **CMSIS-RTOS v2**) : coche `Middleware → FREERTOS`, crée tes tâches dans l'interface, génère.

```

void TacheCapteur(void *arg) {
    for (;;) {
        mesure_t m = lire_bme280();
        osMessageQueuePut(fileMesures, &m, 0, 0);
        osDelay(100); // 100 ms, CPU rendu au système
    }
}

void TacheLog(void *arg) {
    mesure_t m;
    for (;;) {
        if (osMessageQueueGet(fileMesures, &m, NULL, osWaitForever) == osOK)
            printf("T=%d.%d C\r\n", m.t / 10, m.t % 10);
    }
}

```

Notes STM32 : passer la base de temps HAL sur un timer dédié (CubeMX le demande) car SysTick est pris par l'OS ; surveiller la pile de chaque tâche (`uxTaskGetStackHighWaterMark`) ; ISR → uniquement les fonctions `FromISR` .

8. Parcours conseillé : d'Arduino au STM32 en 6 étapes

1. **Blink HAL** + bouton par interruption EXTI (avec anti-rebond logiciel).
2. **UART** : `printf` redirigé, puis réception par interruption dans un **ring buffer** (celui du module 01, enfin utile en vrai) — mini interpréteur de commandes (`led on` , `led off`).
3. **Timers** : PWM qui fait respirer la LED, puis *Input Capture* pour mesurer un HC-SR04, puis base de temps 1 ms qui remplace tous les délais.
4. **ADC + DMA circulaire** : 2 canaux scannés en continu, moyenne glissante, envoi UART toutes les 100 ms — sans une seule attente bloquante.
5. **Refaire la station météo du module 03** : BME280 en I2C, OLED en SPI, architecture en drivers propres (`bme280.c/.h` ...) + FSM.
6. **Version FreeRTOS** de la station : tâches capteur / affichage / console, files et mutex. Puis, pour la culture : refaire l'étape 1 **en registres purs** avec le Reference Manual ouvert.

À l'issue : tu as exactement le profil « firmware junior » que demandent les annonces (STM32, HAL, UART/I2C/SPI, DMA, FreeRTOS, debug SWD).

9. Écosystème et culture

- **CMSIS** : la couche ARM commune (définitions de registres, intrinsics, DSP) — c'est elle qui rend le code portable entre vendeurs.
- **Bootloader système** : tout STM32 peut se flasher par UART/USB (DFU) sans débogueur — broche BOOT0.
- **STM32CubeProgrammer** : outil de flash/lecture, options bytes, protection lecture (RDP).
- Basse conso : modes Sleep/Stop/Standby, réveil par RTC/EXTI — le domaine des séries L.

- Alternatives à la HAL : **libopencm3** (open source), **Zephyr** (RTOS + drivers, très demandé), **Rust/embassy** (module 06 §3 — le blink montré là-bas tourne sur STM32).
 - La même démarche s'applique aux autres 32 bits : ESP-IDF (ESP32), RP2040 SDK — qui a appris STM32+HAL s'adapte en quelques jours.
-

Exercices

1. Chenillard 4 LED cadencé par interruption TIM3, vitesse réglée par potentiomètre sur ADC — zéro `HAL_Delay` dans la boucle.
2. Console UART : réception IT + ring buffer, commandes `pwm <0-100>` et `status`, écho des caractères.
3. Mesure de fréquence : signal carré sur une entrée *Input Capture*, affichage de la fréquence en UART (teste avec le PWM d'une autre broche rebouclée !).
4. `HardFault` volontaire : déréférence un pointeur nul, observe le Fault Analyzer, explique le crash.
5. Étape 6 du parcours (station météo FreeRTOS) — le projet portfolio.

➡ Retour à l'index : [README](#) — ou approfondis avec [Module 06](#) (RTOS, Rust, Linux embarqué).